Cryptography & Security

Rokib Sir's Lectures

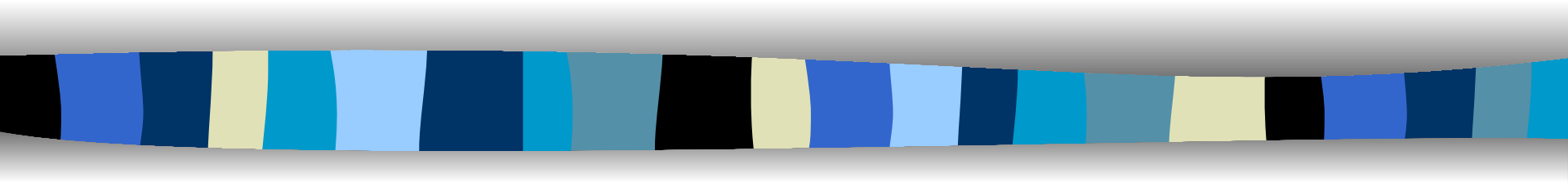
CSE 4115 ? Cryptography & Security

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1	Chapter 1: Security Basics & Cryptography Overview	Page 3
2	Chapter 2: Cryptographic Terminology & Concepts	Page 22
3	Chapter 3: RSA Cryptosystem & Mathematics	Page 113
4	Chapter 4: ElGamal Cryptosystem & Discrete Logarithms	Page 136
5	Chapter 5: ElGamal Public Key Cryptography & Signatures	Page 158
6	Chapter 6: Paillier Cryptosystem & Homomorphic Encryption	Page 170
7	Chapter 7: Cryptographic Hash Functions & MAC	Page 186
8	Chapter 8: Digital Signature Schemes & Verification	Page 194
9	Chapter 9: Properties & Security of Digital Signatures	Page 231

Note1 (Intr1)



Security Problems in Computing

Outline

- Characteristics of computer intrusions
  - Terminology, Types
- Security Goals
  - Confidentiality, Integrity, Availability, …
- Vulnerabilities
  - Hardware, Software, Data, …
- Methods of Defense
  - Encryption, h/w control, s/w control, …

Status of security in computing

- In terms of security, computing is very close to the wild west days.
- Some computing professionals & managers do not even recognize the value of the resources they use or control.
- In the event of a computing crime, some companies do not investigate or prosecute.

Characteristics of Computer Intrusion

- A **computing system**: a collection of hardware, software, data, and people that an organization uses to do computing tasks
- Any piece of the computing system can become the **target** of a computing crime.
- The **weakest point** is the most serious vulnerability.
- The **principle of easiest penetration**

Security Breaches - Terminology

- Exposure
  - a form of possible loss or harm
- Vulnerability
  - a weakness in the system
- Attack
- Threats
  - Human attacks, natural disasters, errors
- Control – a protective measure
- Assets – h/w, s/w, data

Types of Security Breaches

- Interruption
  - Example: DOS (Denial of Service)
- Interception
  - Peeping eyes
- Modification
  - Change of existing data
- Fabrication
  - Addition of false or spurious data

Security Goals

- **Confidentiality**
  - The assets are accessible only by authorized parties.
- **Integrity**
  - The assets are modified only by authorized parties, and only in authorized ways.
- **Availability**
  - Assets are accessible to authorized parties.

Computing System Vulnerabilities

- Hardware vulnerabilities
- Software vulnerabilities
- Data vulnerabilities
- Human vulnerabilities ?

Software Vulnerabilities

- Destroyed (deleted) software
- Stolen (pirated) software
- Altered (but still run) software
  - Logic bomb
  - Trojan horse
  - Virus
  - Trapdoor
  - Information leaks

Data Security

- The **principle of adequate protection**
- Features
  - Confidentiality: preventing unauthorized access
  - Integrity: preventing unauthorized modification (e.g., salami attack)
  - Availability: preventing denial of authorized access

Other Exposed Assets

- Storage media
- Networks
- Access
- Key people

People Involved in Computer Crimes

- Amateurs
- Crackers
- Career Criminals

Methods of Defense

- Encryption
- Software controls
- Hardware controls
- Policies
- Physical controls

Encryption

- At the heart of all security methods
- Confidentiality of data
- Some protocols rely on encryption to ensure availability of resources.
- Encryption does not solve all computer security problems.

Software controls

- Internal program controls
- OS controls
- Development controls
- Software controls are usually the 1st aspects of computer security that come to mind.

Policies

- Policy controls can be simple but effective
  - Example: frequent changes of passwords
- Legal and ethical controls
  - Gradually evolving and maturing

Principle of Effectiveness

- Controls must be used to be effective.
  - Efficient
    - Time, memory space, human activity, …
  - Easy to use
  - appropriate

Overlapping Controls

- Several different controls may apply to one potential exposure.
  - H/w control
  - S/w control
  - Data control

Summary

- A very high-level overview
- The principle of easiest penetration
- Effective control
- Overlapping control

Key terms

- **Exposure**
- **Vulnerability**
- **Attack**
- **Threat**
- **Control**
- **Major assets of computing:**
  - Hardware, Software, Data

Attacks, Services and Mechanisms

- **Security Attack:** Any action that compromises the security of information.
- **Security Mechanism:** A mechanism that is designed to detect, prevent, or recover from a security attack.
- **Security Service:** A service that enhances the security of data processing systems and information transfers. A security service makes use of one or more security mechanisms.

Security Attacks

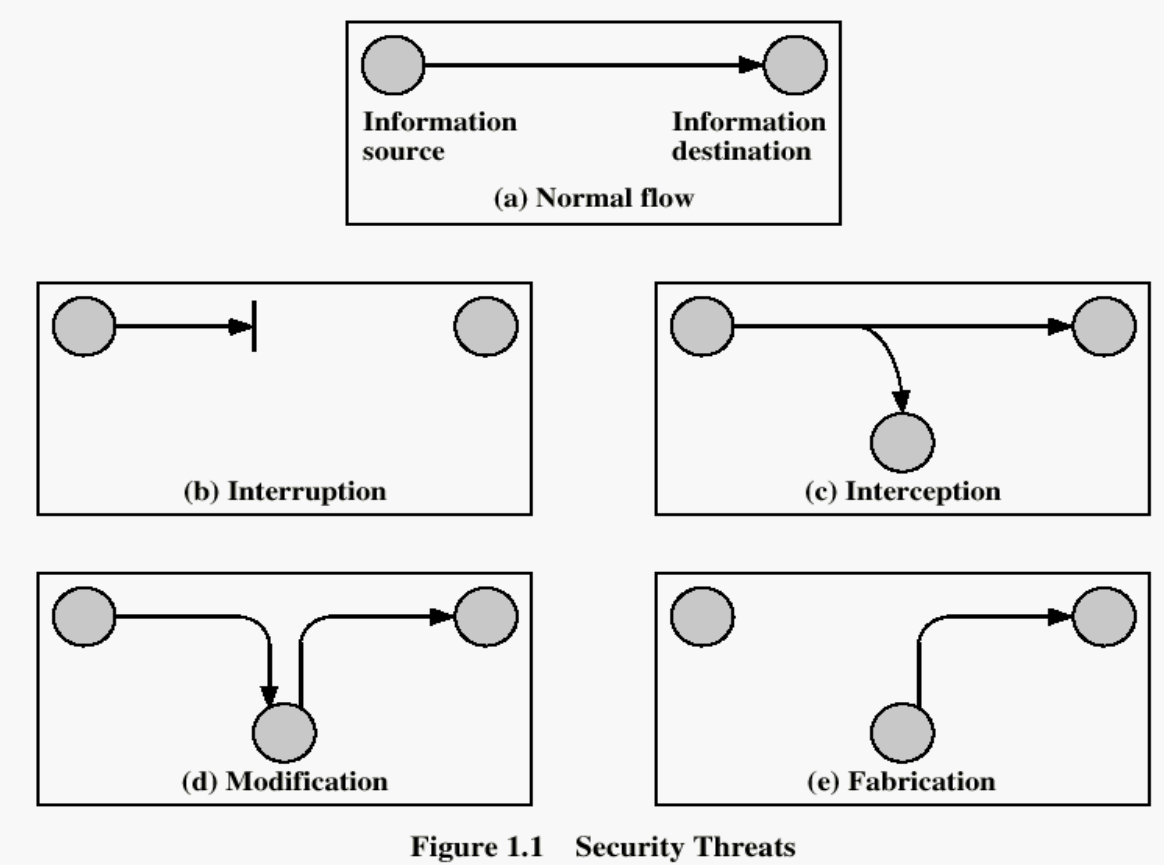


Figure 1.1 Security Threats

Security Attacks

- **Interruption:** This is an attack on availability
- **Interception:** This is an attack on confidentiality
- **Modification:** This is an attack on integrity
- **Fabrication:** This is an attack on authenticity

Goals of security

- **Confidentiality**
  - Secrecy, privacy
- **Integrity**
- **Availability**
  - Denial of service
- **Other goals**
  - Authenticity, non-repudiation, authority, utility, accountability, etc.

Vulnerabilities

- **Hardware**
  - Interruption (Denial of service)
  - Interception (Theft)
- **Software**
  - Interruption (deletion)
  - Interception
  - Modification
  - Logic bomb, Trojan Horse, virus, Trapdoor, Leak of information

Vulnerabilities

- **Data**
  - Interruption (loss)
  - Interception
  - Modification
  - Fabrication
- **Other assets**
  - Storage Media
  - networks
  - Access
  - Key people

People Involved

- **Amateurs**
- **Crackers**
- **Career criminals / Hackers**

Methods of Defense

- **Encryption**
- **Software controls** (access limitations in a data base, in operating system protect each user from other users)
  - Internal program controls
  - Operating system controls
  - Development controls
- **Hardware controls** (e.g. smart cards)

Methods of Defense

- **Policies** (frequent changes of passwords)
- **Physical controls**

Effectiveness of controls

- **Awareness of problem**
- **Likelihood of use**
- **Overlapping controls**
- **Periodic review**

Outline

- **Encryption**
  - Basics (Terminology)
  - Histroy
  - Encryption Principles
  - Classifications
  - Key Distribution
- **Protocols**
  - Security/cryptographic protocols
  - Authentication
  - Digital Signatures

Outline

- **Human controls in security**
  - Administration of security
  - Human controls, law, ethics etc.
  - The future?

Terminologies

- **Plaintext:** Message or data which are in their normal, readable (not crypted) form.
- **Encryption:** Encoding the contents of the message in such a way that hides its contents from outsiders.
- **Ciphertext**: The encrypted message

Terminologies

- **Decryption:** The process of retrieving the plaintext back from the ciphertext.

- **Key:** Encryption and decryption usually make use of a key, and the coding method is such that decryption can be performed only by knowing the proper key.

Terminologies

- **Cryptography** is the art or science of keeping messages secret. It deals with all aspects of secure messaging, authentication, digital signatures, electronic money, and other applications.

- **Cryptosystems:** A cryptographic system (cryptosystem) consists of a pair of data transformations, namely encryption and decryption.

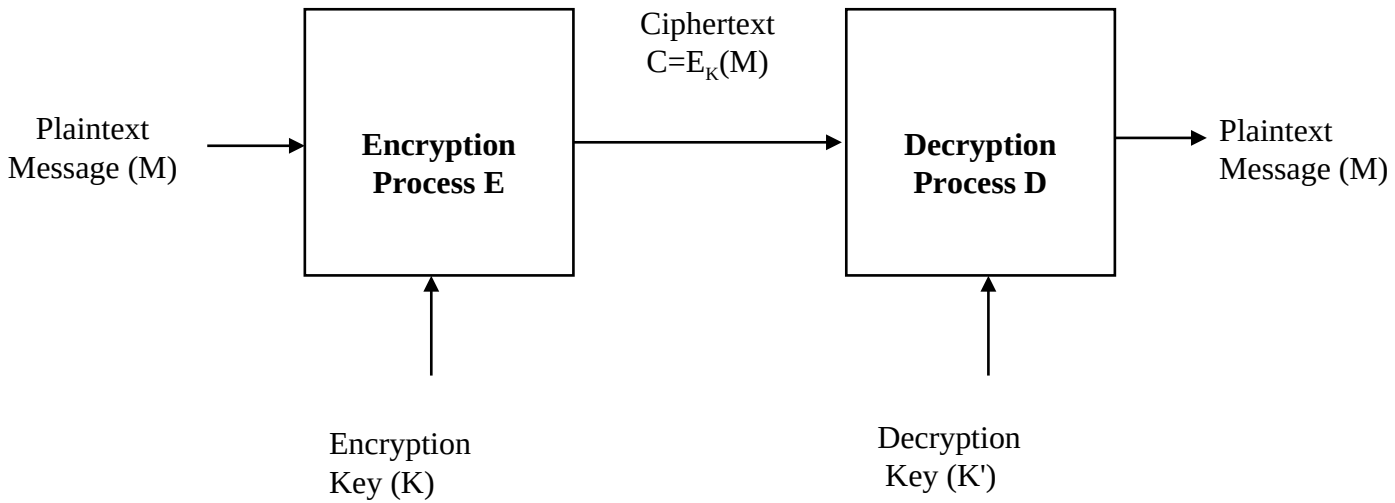
Terminologies

- **Cryptanalysis:** The art of **breaking** ciphers, i.e. retrieving the plaintext without knowing the proper key.
- **Cryptographers:** People who do cryptography
- **Cryptanalysts:** practitioners of cryptanalysis

Conventional Cryptosystem Principles

- **An cryptosystem has the following five ingredients:**
  - Plaintext
  - Encryption algorithm
  - Secret Key
  - Ciphertext
  - Decryption algorithm
- **Security depends on the secrecy of the key, not the secrecy of the algorithm**

Conventional Cryptosystem Principles

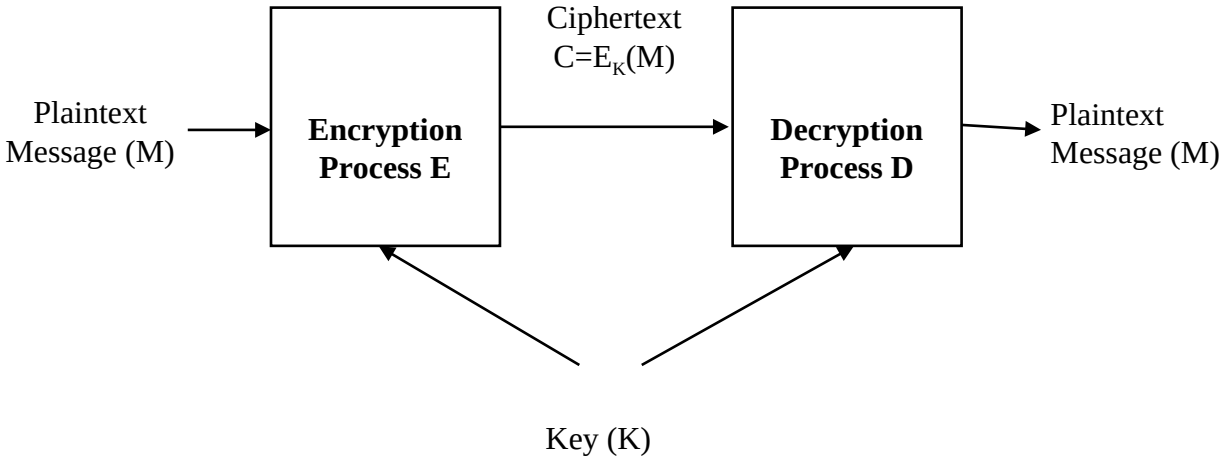


Classifications

- **Classification of cryptosystems**
  - Symmetric cryptosystems
  - Asymmetric cryptosystems

Symmetric Cryptosystem

- The same key is used for both encryption and decryption purposes



Symmetric Cryptosystem

- Examples of symmetric cryptosystem are Data Encryption Standard (DES)

- Problem : How do we distribute the key securely?

Key Distribution

- A key could be selected by A and physically delivered to B.
- A third party could select the key and physically deliver it to A and B.
- If A and B have previously used a key, one party could transmit the new key to the other, encrypted using the old key.

Key Distribution

- If A and B each have an encrypted connection to a third party C, C could deliver a key on the encrypted links to A and B.

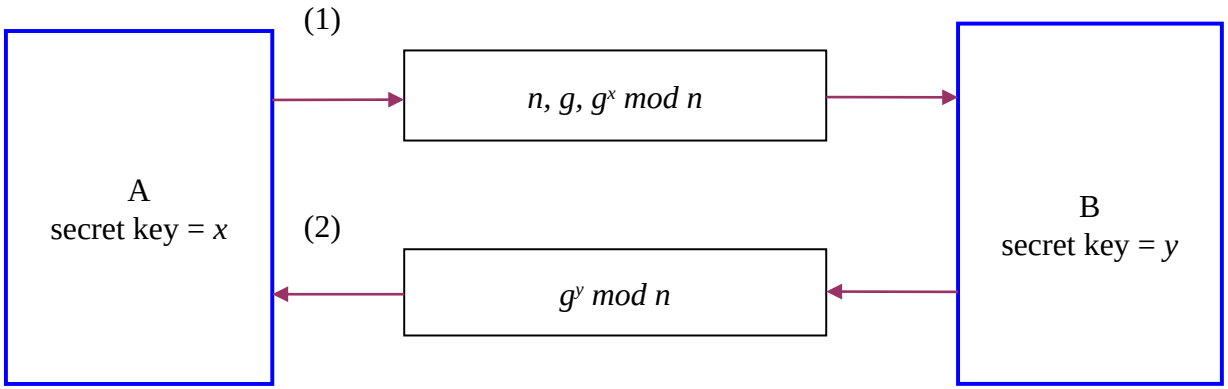
- **Session key:**
  - Data encrypted with a one-time session key.At the conclusion of the session the key is destroyed

Key Distribution

- **Permanent key:**
  - Used between entities for the purpose of distributing session keys

- **Protocol:**
  - Defines the detail formats of messages sent from one entity to the another to accomplish a job

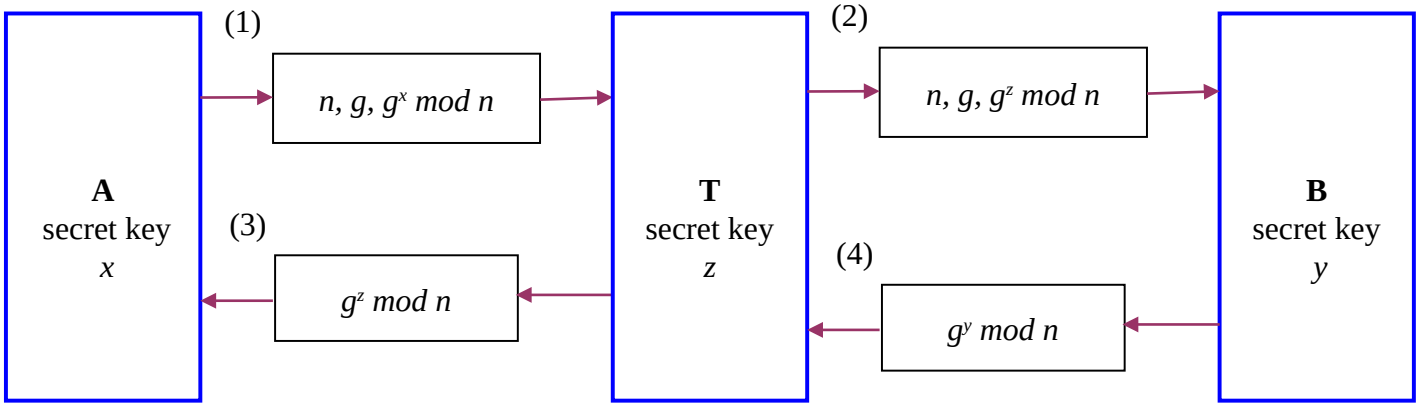
A Key Exchange Protocol



- *n* and *g* are large prime numbers. Both n and g are made public
- A picks a large (e.g. 512 bit) number *x* and keeps it secret. Similarly B picks a large secret number *y*.
- At the end of the protocol, both entities A and B end up possessing the same secret key $g^{xy} \bmod n$

A Key Exchange Protocol

- *Protocols are difficult to design*



A computes
$(g^z \bmod n)^x$
=
$g^{zx} \bmod n$

T computes
$(g^x \bmod n)^z = g^{xz} \bmod n$
and
$(g \bmod n)^z = g^{yz} \bmod n$

B computes
$(g^z \bmod n)^y$
=
$g^{yz} \bmod n$

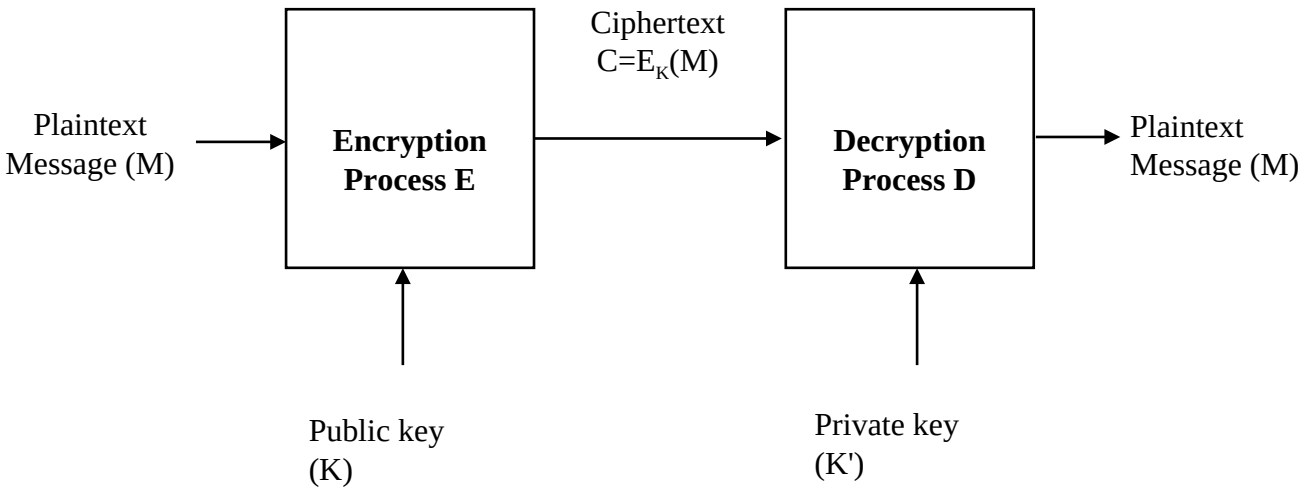
Assymmetric Cryptosystem

- Different keys are used for encryption and decryption purposes.
- The pair of keys are mathematically related and consist of a public key that can be published without doing harm to the system's security and a private key that is kept secret.
- Also known as public key cryptosystems

Assymmetric Cryptosystem

- The public key is used for encryption purposes and lies in the public domain.
- Anybody can use the public key to send an encrypted message.
- The private key is used for decryption purposes and remains secret.
- An example of a public cryptosystem is the RSA cryptosystem.

Assymmetric Cryptosystem



Encyption – can it be broken?

- Theoretically, it is possible to devise unbreakable cryptosystems
- However, practical cryptosystems almost always are breakable, given adequate time and computing power
- The trick is to make breaking a cryptosystem hard enough for the intruder

Types of Ciphers

- Ciphers can be broadly classified into the following two categories depending upon whether
  - (i) a symbol of plaintext is immediately converted into a symbol of ciphertext (Stream Ciphers)
  - (ii) or a group of plaintext symbols are converted as a block into a group of ciphertext symbols (Block Ciphers)

Stream Ciphers

- A symbol of plaintext is immediately converted into a symbol of ciphertext
- **Advantages**
  - Speed of transformation
  - Low error propagation
- **Disadvantages**
  - Low diffusion
  - Susceptible to malicious insertions and modifications

Block Ciphers

- A group of plaintext symbols are converted as a block into a group of ciphertext symbols
- **Advantages**
  - Diffusion
  - Immunity to insertions
- **Disadvantages**
  - Slowness of encryption
  - Error propagation

General Types of Ciphers

- **Substitution ciphers**
  - Letters of the plaintext messages are replaced with other letters during the encryption

- **Transposition ciphers**
  - The order of plaintext letters is rearranged during encryption

General Types of Ciphers

- **Product ciphers**
  - Combine two or more ciphers to enhance the security of the cryptosystem

Trends

- **Block size:** larger block sizes mean greater security
- **Key Size:** larger key size means greater security
- **Number of rounds:** multiple rounds offer increasing security

Monoalphabetic Substitution Ciphers

- **Caesar cipher**

$$c_i = E(p_i) = p_i + 3 \bmod 26$$

```
Plaintext: A B C D E F G H I J K L M N O P Q R S T U V
    W X Y Z
Ciphertext: d e f g h i j k l m n o p q r s t u v w x
    y z a b c
```

- **Example**

```
Plaintext: CRYPTOGRAPHY IS GREAT FUN
Ciphertext: fubswrjudskb lv juhdw ixq
```

Polyalphabetic Substitution Ciphers

- Flatten the frequency distribution of letters by combining high and low distributions

- **Example:**

Plaintext: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Ciphertext1: a d g j m p s v y b e h k n q t w z c f i l o r u x

Ciphertext2: n s x c h m r w b g l q v a f k p u z e j o t y d i

Plaintext: VIGENERE TABLEAUX

Ciphertext: lbshnhzh fndqmniy

Transposition Ciphers

- Rearrangement of the letters or a message

Columnar transposition

Plaintext	Ciphertext
W H Y D O	welrnel
E S I T A	hswatta
L W A Y S	yiaihhn
R A I N I	dtyneed
N T H E N	oasinrs
E T H E R	
L A N D S	

Characteristics of good cipher

- **Shannon characteristics**
  - The amount of secrecy should determine the amount of labor appropriate for the encryption and decryption
  - The set of keys and encryption algorithm should be free of complexity
  - The implementation of the process should be as simple as possible

Characteristics of good cipher

- Errors in encryption should not propagate and cause corruption of further information in the message.
- Ciphertext size should not be larger than plaintext

- **Confusion**
  - The change in ciphertext triggered by an alteration in the plaintext should be unpredictable

Characteristics of good cipher

- **Diffusion**
  - Change in the plaintext should affect many parts of the ciphertext

- **Other issues**
  - Perfect secrecy vs. Effective secrecy
  - Redundancy of languages
  - Unicity distance

Methods of attack

- **Ciphertext-only attack**
  - The attacker gets a ciphertext and tries to find the corresponding plaintext.

- **Known-plaintext attack**
  - The attacker has some plaintext and its matching ciphertext. The task is to find a key corresponding to this match.

Methods of attack

- **Chosen-plaintext attack**
  - Here, the attacker selects a plaintext and ciphers it using the cryptotechinque he attacks. The plaintext may be chosen to ease the task of key finding.

Application of Cryptography

- Confidentiality
- Authentication
- Message Integrity
- Digital Signature

Confidentiality

- Confidentiality of a message can be achieved by encrypting it with a key (symmetric/asymmetric).
- Only the authorized recipients of the message possessing the decryption can decrypt the message.
- It will become difficult for an intruder to see the content of the message in the absence of the appropriate key.

Authentication

- Authentication is the process of reliably verifying the identity of a distributed entity amidst threats arising from the environment.

- In a computer system there are generally three different levels of authentication that are involved as given below
  - User Authentication

Authentication

- Authentication of a distributed entity (e.g. remote computer, smart card, remote process etc.)

- Authentication of the system to the entity - System Authentication.

Authentication

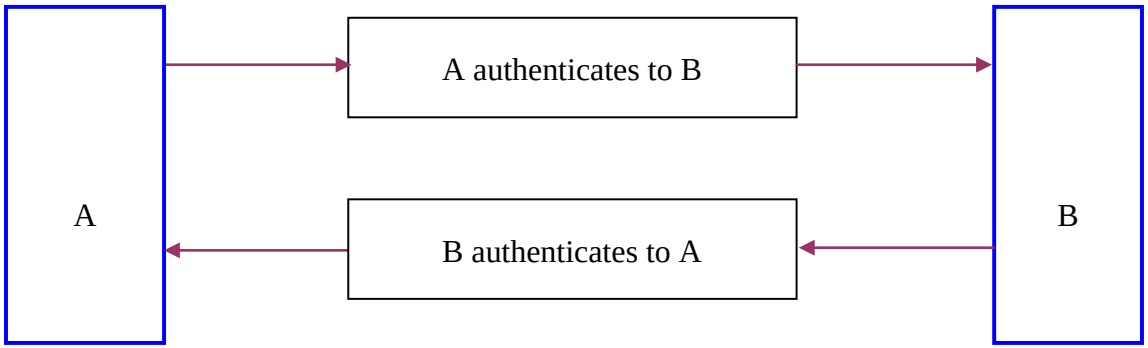
- Most of the mutual authentication protocol addresses the following two different issues:
  - Authentication of distributed entities.
  - Establishment of a random session key between the authenticated entities

Authentication

- For any claimant entity to authenticate itself to a verifier entity two different strategies exist namely:
  - Direct authentication
  - Authentication via a trusted third party

Authentication

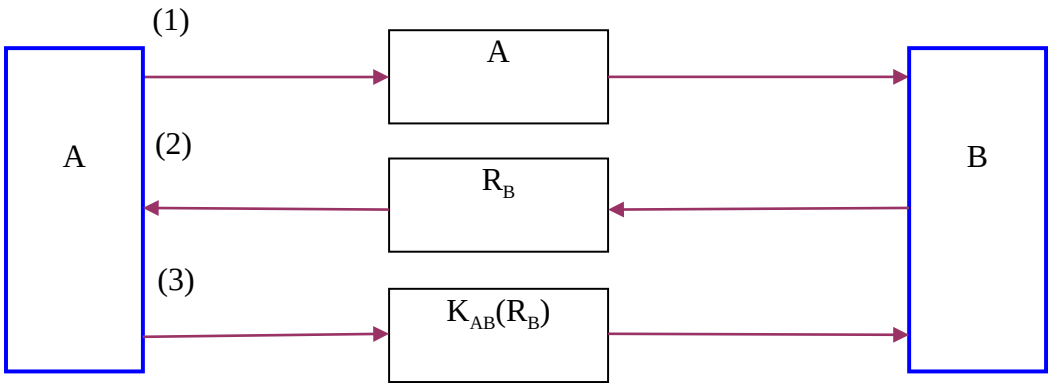
- **Direct authentication**



- **Limitations** - Key management is relatively complex, e.g. for a distributed entity to communicate securely with *n* other entities, it needs to maintain a minimum of *n* keys.

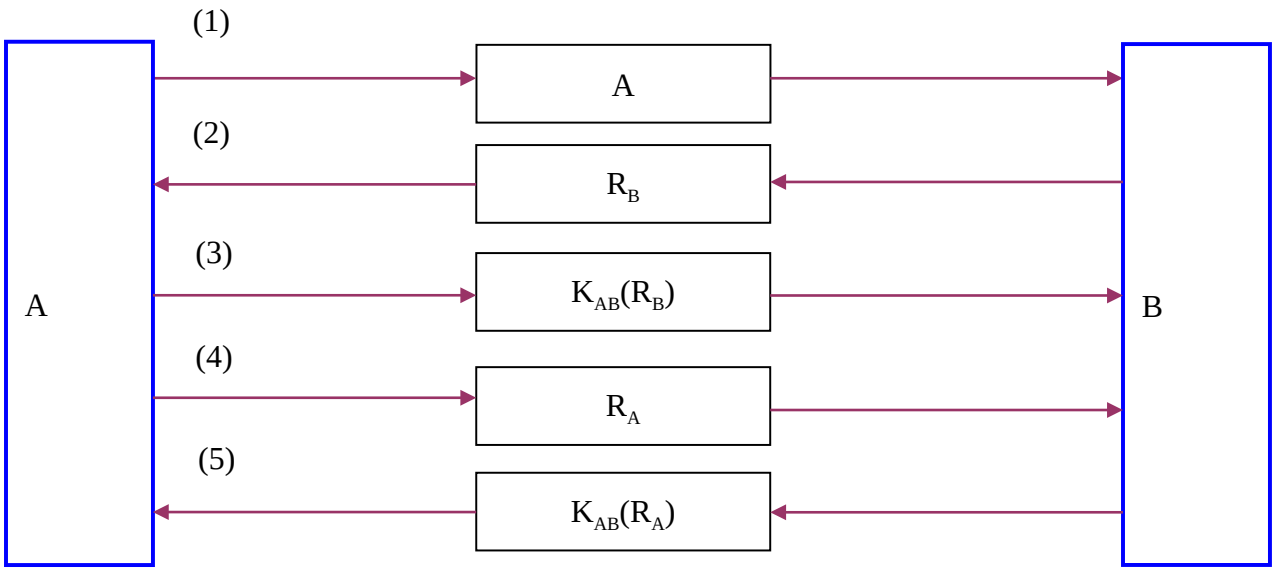
Authentication

- Example – Unidirectional Authentication



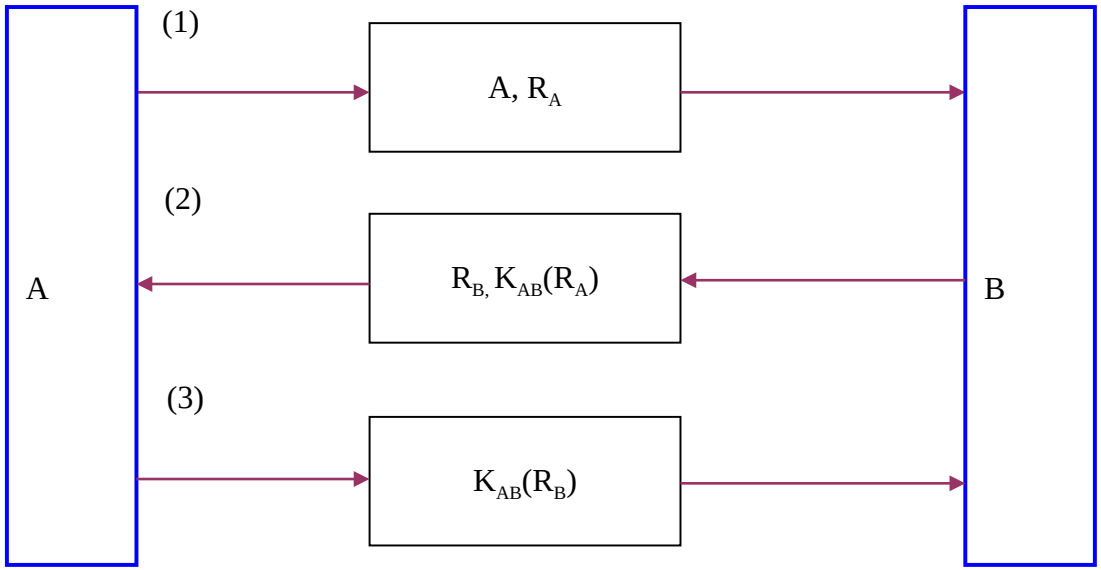
Authentication

- Example – Mutual Authentication



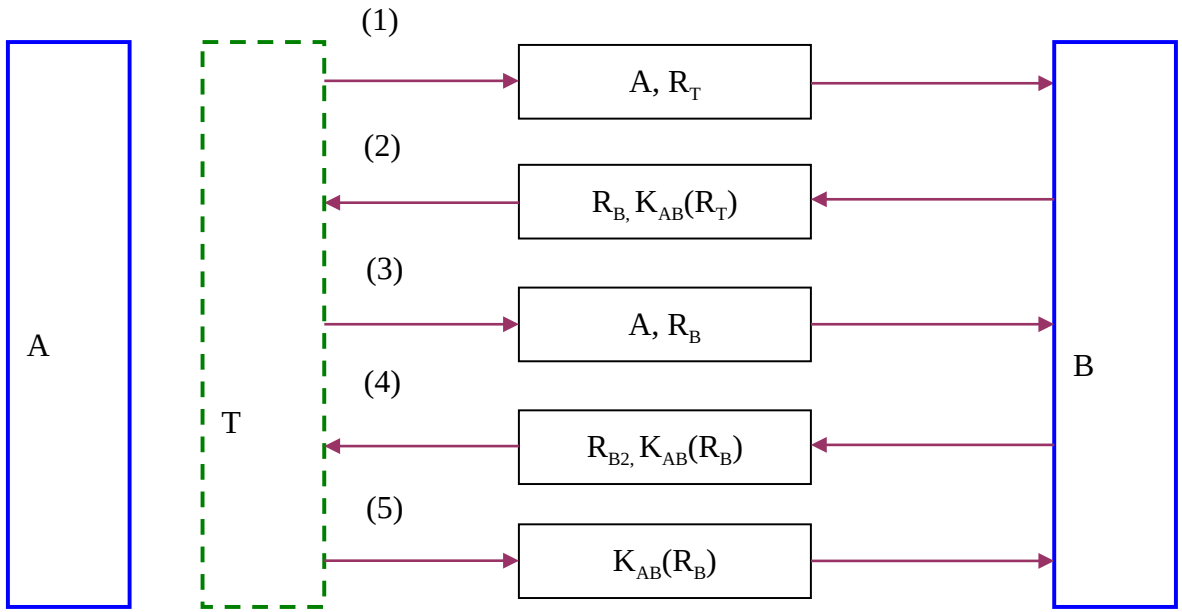
Authentication

- Example – Optimized Mutual Authentication



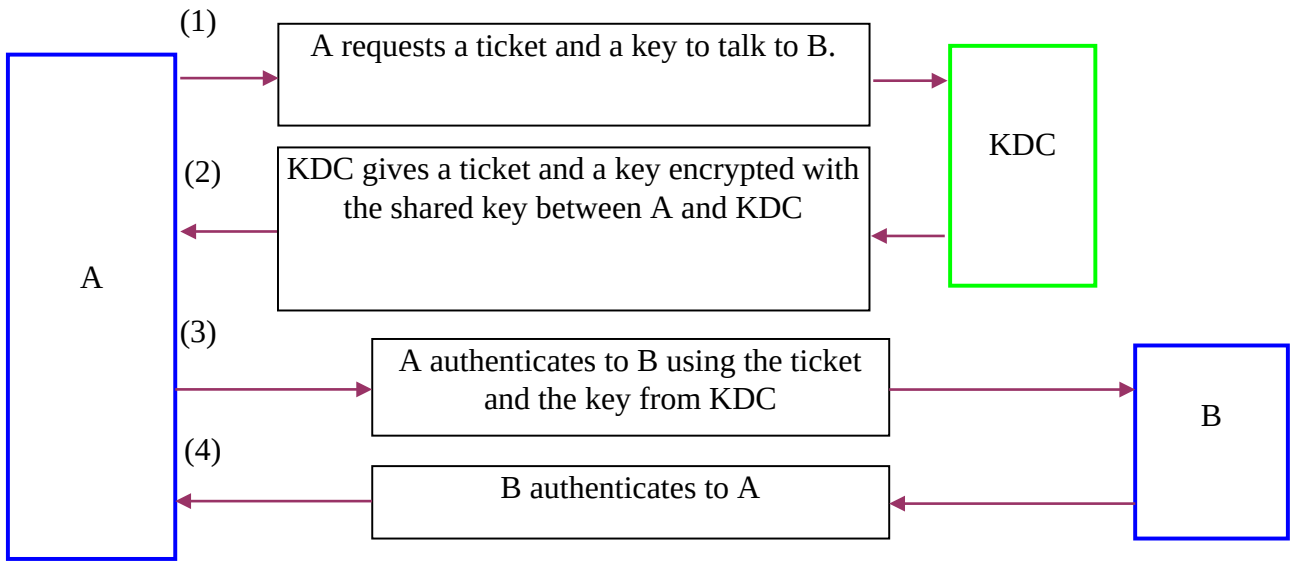
Authentication

- Problem !!!



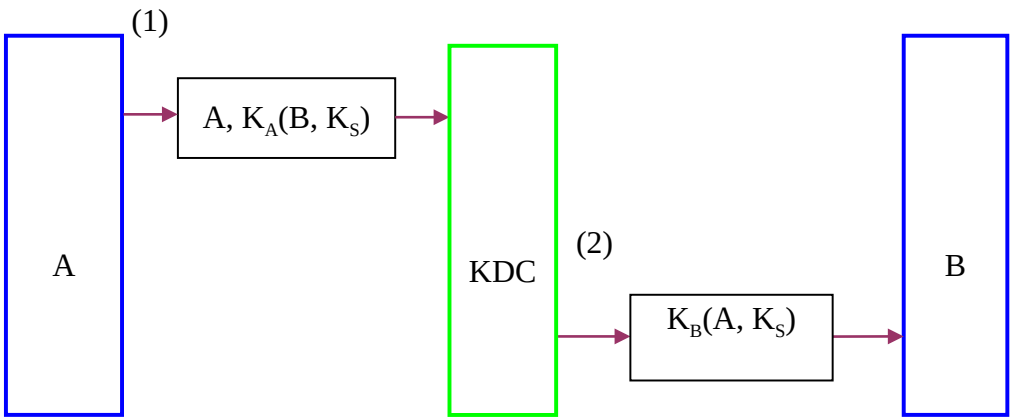
Authentication

- **Authentication via a trusted third party**



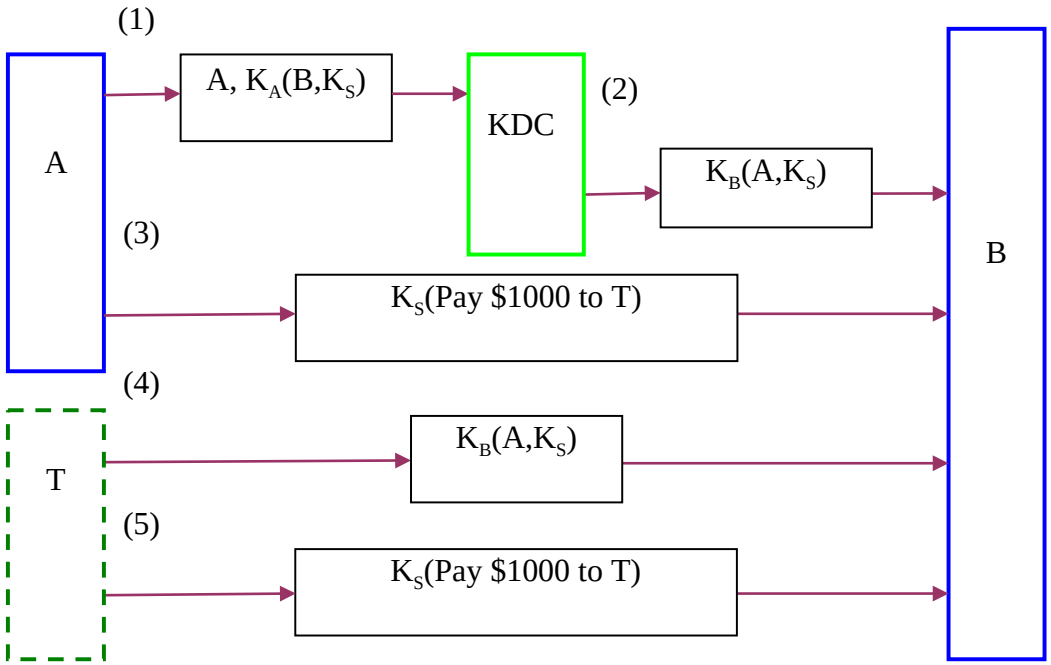
Authentication

- Example



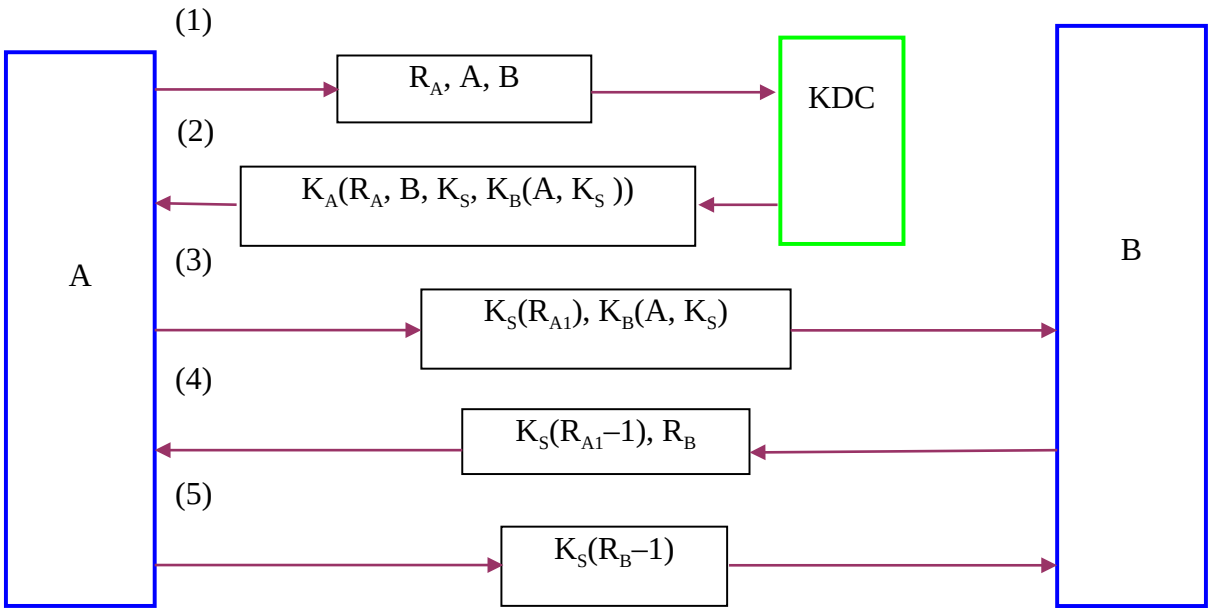
Authentication

- Problem – Replay attack



Authentication

- Another Example ??



Authentication

- Problem
  - Old session keys can be valuable. If T can manage to get hold of an old session key, it can launch a successful **replay attack** by replaying the sequence from message (3) and convince B that it is A.
  - If the key shared between A and the KDC is ever compromised, the consequences can be drastic. T can use the key to obtain session keys to talk with anyone.

Authentication

- **Message**
  - Authentication Protocols are very hard to design.

Message Authentication

- **Objective:**
  - Contents have not been altered
  - A hash function is used

- Hash Functions
  - A hash function is a one way function that maps values from a large domain into a comparatively small range known as a digest.

Message Authentication

- **Properties of a HASH function H :**
  - H can be applied to a block of data at any size
  - H produces a fixed length output
  - H(x) is easy to compute for any given x.
  - For any given block x, it is computationally infeasible to find x such that H(x) = h
  - For any given block x, it is computationally infeasible to find with H(y) = H(x).
  - It is computationally infeasible to find any pair (x, y) such that H(x) = H(y)

Message Authentication

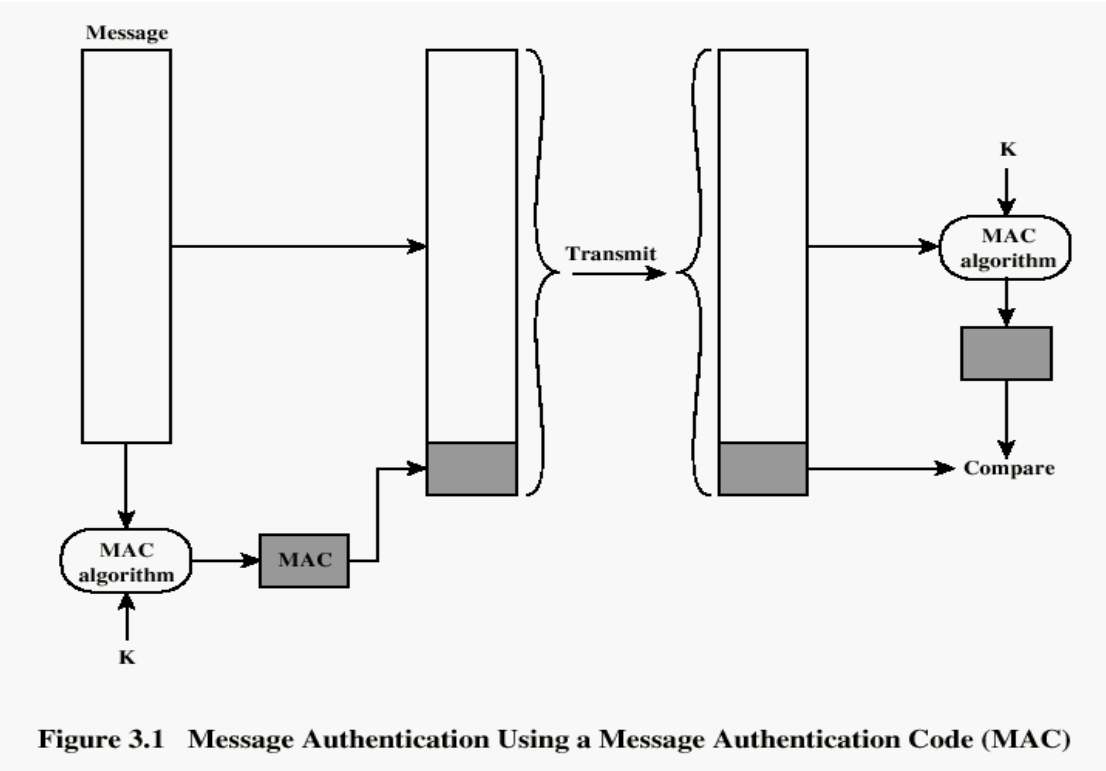


Figure 3.1 Message Authentication Using a Message Authentication Code (MAC)

Digital Signature

- A message can be attached a digital signature to guarantee authenticity, integrity and non-repudiation.
- Asymmetric Cryptography is used.
- A digital signature is a block of data that is generated by the sender of a message using his/her secret key. The public key of the user is later used by the receiver to verify whether the message was *signed* by that particular user.

Digital Signature

- The following are the features of digital signature
  - Verification of a correct signature will succeed
  - Modification of a signed message will be detected
  - Signature will not help divulge signer’s private key
  - Only parties in the possession of a secret key will be able to produce a valid signature

Software Security

- **Why are software flawed?**
  - Controls apply at individual program or programmer level
  - Software engineering techniques evolve much faster than security techniques
  - Malicious software vs. accidental errors

Malicious code

- **Type** **Characteristics**

  Virus Attaches itself to programs and propagates copies of itself to other programs

  Trojan horse Contains unexpected functionality

  Logic bomb Triggers action when a condition occurs

  Time bomb Triggers action at a certain time

  Trapdoor Allows unauthorized access to functionality

Malicious code

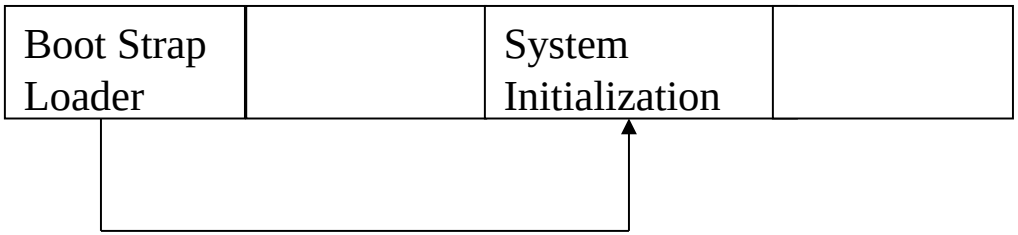
Type	Characteristics
Worm	Propagates copies of itself through a network
Rabbit	Replicates without limit to exhaust resources

”Good viruses”

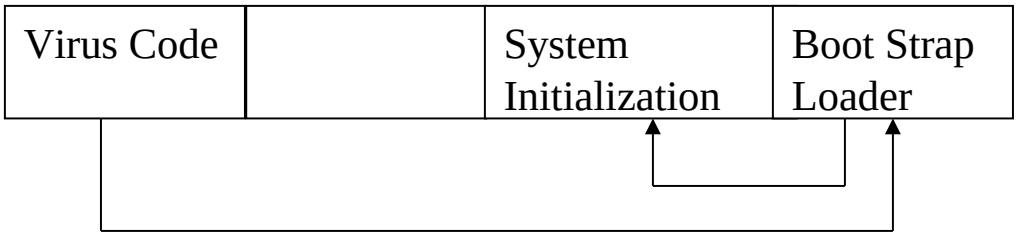
- Are hard to detect
- Are hard to destroy
- Spread widely
- Can re-infect cleaned files
- Are easy to create
- Are machine independent

Hiding places

- Boot sector



Normal Process



Infection

Hiding places

- Memory- resident viruses
- Macro, library etc. viruses

Effects and causes

Effect	How caused?
Attach to executable Program	• Modify file directory • Write to executable file
Attach to data or control • Rewrite data • Append to data • Append data to itself	• Modify directory

Effects and causes

Effect	How caused?
Remain in memory	• Intercept interrupts and modify handlers
Infect disks	• Intercept interrupt • Intercept OS call

- Modify system file
- Modify ordinary executables

Effects and causes

Effect	How caused?
Spread infection • Infect system program • Infect ordinary program • Infect data that controls ordinary programs	• Infect boot sector

How to prevent infections?

- Make sure you know the source of software
- Test new software on an isolated computer
- Make backups of bootable disks, store safely
- Keep backups of system files
- Use detectors
- Be careful with macro scripts

Outline

- Network threats
- Network controls
- Firewalls
- Internet security

Network threats

- Causes of security problems:
  - Sharing of resources and workload
  - Complexity of systems and interconnection mechanisms
  - Unknown security perimeter
  - Multiple points of attacks
  - Anonymity of attackers
  - Unknown access paths to resources

What could be attacked?

- *local nodes* connected via *local communications links* to a *local area network* which also has *local data storage*, *local processes* , and *local devices*. The LAN is also connected to a *network gateway* that gives access via *network communications links* to *network control resources* , *network routers*, and *network resources*, such as databases.

What can an attacker do?

- Intercept data in transit
- Modify data in transit
- Gain unauthorized access to programs or data in remote hosts
- Modify programs or data in remote hosts
- Insert communications
- Replay previous communication
- Block selected traffic
- Block all traffic
- Run a program at a remote host

By what means?

- Wiretapping
- Impersonation
- Message confidentiality violations
- Message integrity violations
- Hacking
- Code integrity violations
- Denial of service

Wiretapping

- Passive vs. active wiretapping
  - Cable
  - Microwave
  - Satellite communications
  - Optical fibre

Message confidentiality violations

- Mis-delivery
- Exposure in processing systems

Message integrity violations

- Change content of a message
- Change part of the content of a message
- Replace a message
- Reuse an old message
- Change the apparent source of a message
- Redirect a message
- Destroy or delete a message

Hacking

- hacker vs. cracker
- Hacking tools
- Automated attacks
- Distributed automated attacks
- Are they a real threat?

Code integrity violations

- User is typically unaware of the content of the downloaded file
- File downloading may happen without user’s permission

Denial of service

- Connectivity
- Flooding
- Routing problems
- Disruption of service

Firewalls

- In the good ol’ days, cities were protected by thick walls, and houses were separated from each other by firewalls that prevented of, for example, spread of fire throughout the city.

- Single point of control where network traffic is examined, could help in the maintenance of security

Firewalls

- Physical world analogies:
  - Passport (and visa) checking at borders
  - Apartments are often locked at the entrance in addition to each door
- Properties:
  - All traffic from inside to outside, and vice versa, must pass through a firewall

Firewalls

- Only authorized traffic, as defined by the local security policy, will be allowed to pass
- The firewall itself is immune to penetration

Public Key Cryptography and the RSA Algorithm

Private-Key Cryptography

- traditional **private/secret/single key** cryptography uses **one** key
- Key is shared by both sender and receiver
- if the key is disclosed communications are compromised
- also known as **symmetric**, both parties are equal
  - hence does not protect sender from receiver forging a message & claiming is sent by sender

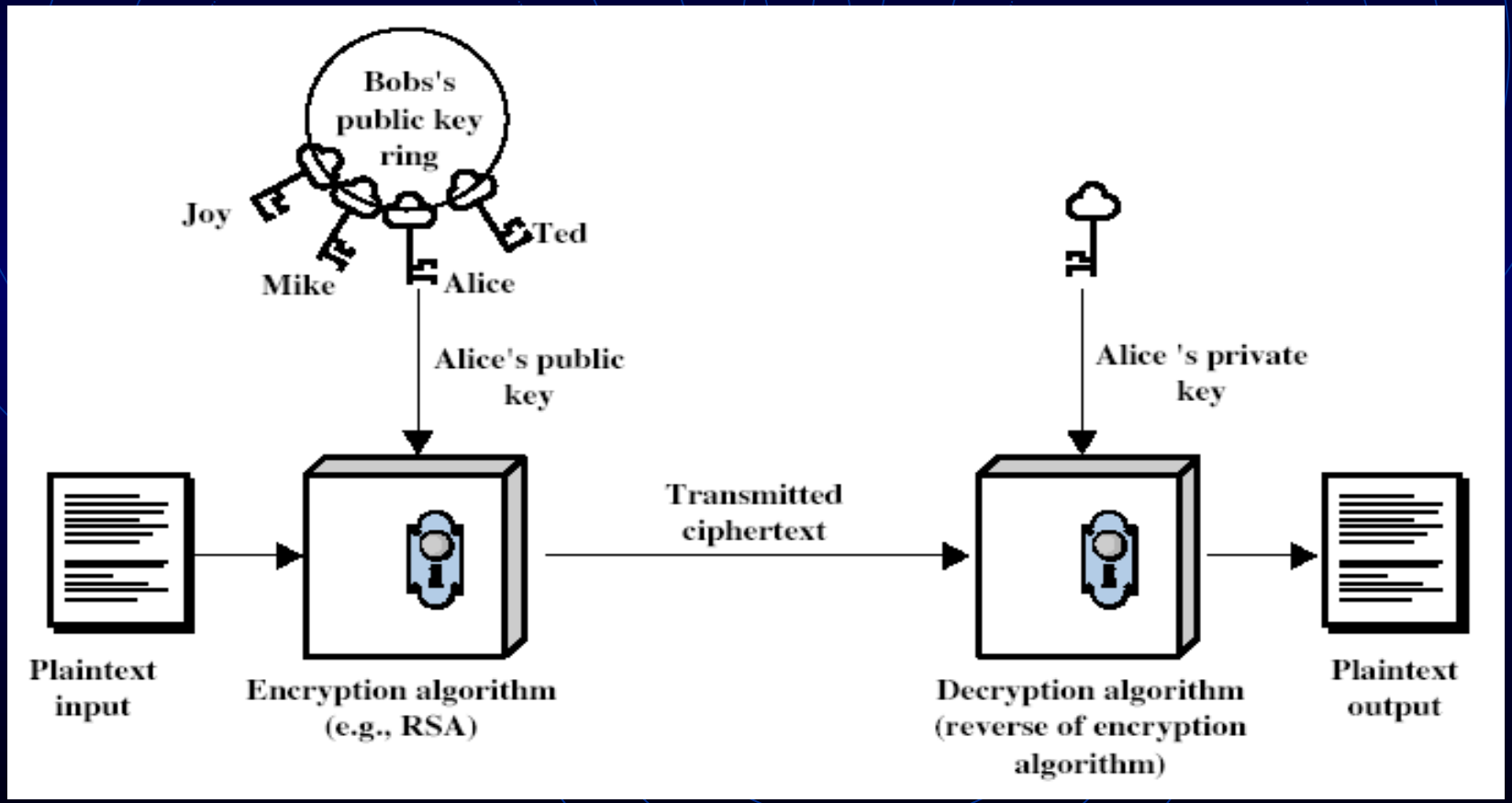
Public-Key Cryptography

- probably most significant advance in the 3000 year history of cryptography
- uses **two** keys – a public key and a private key
- **asymmetric** since parties are **not** equal
- uses clever application of number theory concepts to function
- complements **rather than** replaces private key cryptography

Public-Key Cryptography

- **public-key/two-key/asymmetric** cryptography involves the use of **two** keys:
  - a **public-key**, which may be known by anybody, and can be used to **encrypt messages**, and **verify signatures**
  - a **private-key**, known only to the recipient, used to **decrypt messages**, and **sign** (create) **signatures**
- is **asymmetric** because
  - those who encrypt messages or verify signatures **cannot** decrypt messages or create signatures

Public-Key Cryptography



Why Public-Key Cryptography?

- developed to address two key issues:
  - **key distribution** – how to have secure communications in general without having to trust a KDC with your key
  - **digital signatures** – how to verify a message comes intact from the claimed sender
- public invention due to Whitfield Diffie & Martin Hellman at Stanford U. in 1976
  - known earlier in classified community

Public-Key Characteristics

- Public-Key algorithms rely on two keys with the characteristics that it is:
  - computationally infeasible to find decryption key knowing only algorithm & encryption key
  - computationally easy to en/decrypt messages when the relevant (en/decrypt) key is known
  - either of the two related keys can be used for encryption, with the other used for decryption (in some schemes)

Public-Key Cryptosystems

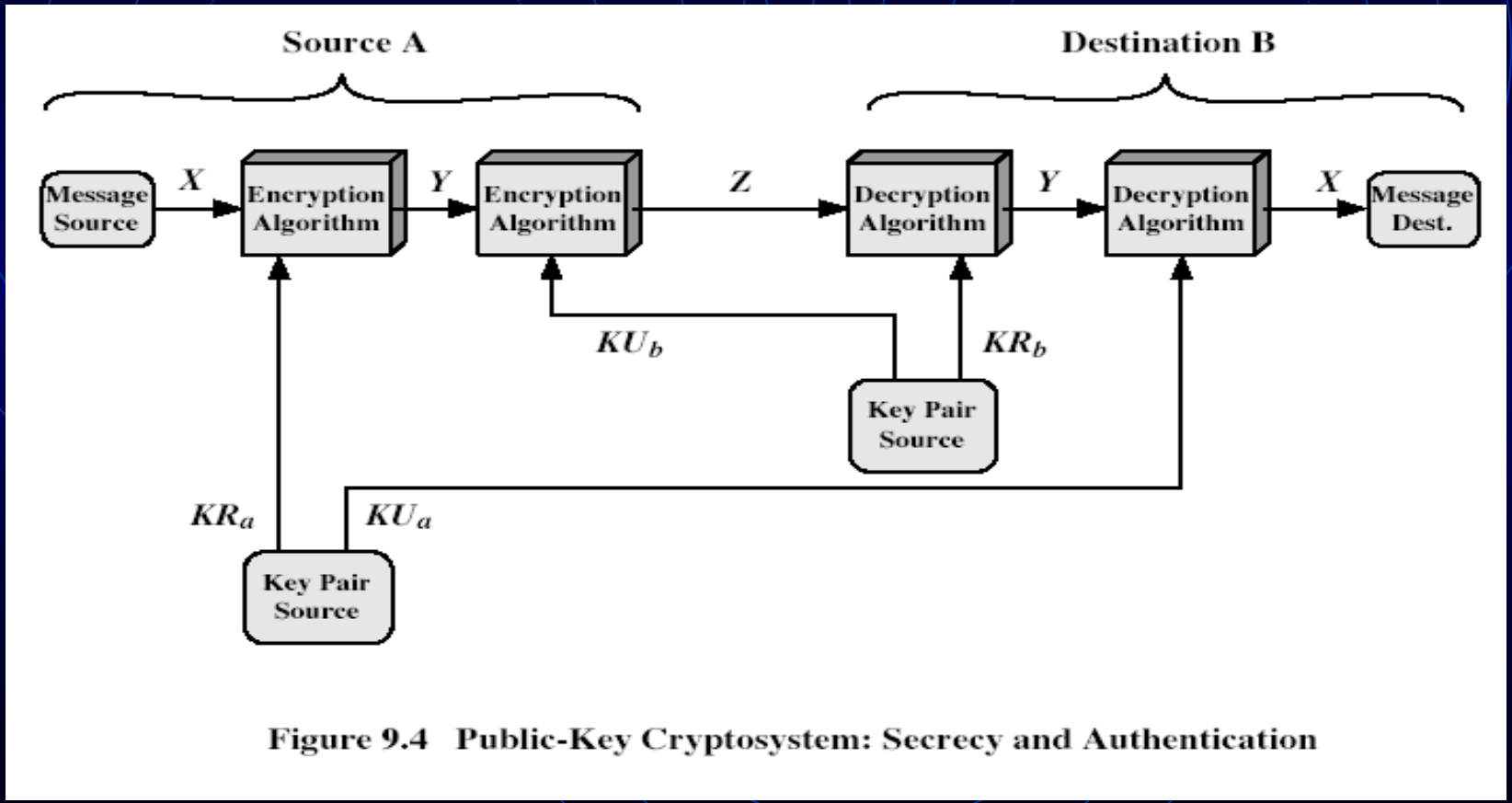


Figure 9.4 Public-Key Cryptosystem: Secrecy and Authentication

Public-Key Applications

- can classify uses into 3 categories:
  - **encryption/decryption** (provide secrecy)
  - **digital signatures** (provide authentication)
  - **key exchange** (of session keys)
- some algorithms are suitable for all uses, others are specific to one

Security of Public Key Schemes

- like private key schemes brute force **exhaustive search** attack is always theoretically possible
- but keys used are too large (>512bits)
- security relies on a **large enough** difference in difficulty between **easy** (en/decrypt) and **hard** (cryptanalyse) problems
- more generally the **hard** problem is known, its just made too hard to do in practise
- requires the use of **very large numbers**
- hence is **slow** compared to private key schemes

RSA

- by Rivest, Shamir & Adleman of MIT in 1977
- best known & widely used public-key scheme
- based on exponentiation in a finite (Galois) field over integers modulo a prime
  - nb. exponentiation takes $O((\log n)^3)$ operations (easy)
- uses large integers (eg. 1024 bits)
- security due to cost of factoring large numbers
  - nb. factorization takes $O(e^{\log n \log \log n})$ operations (hard)

RSA Key Setup

- each user generates a public/private key pair by:
- selecting two large primes at random - p, q
- computing their system modulus $N=p.q$
  - note $ø(N)=(p-1)(q-1)$
- selecting at random the encryption key e
    - where $1<e<ø(N)$, $gcd(e,ø(N))=1$
- solve following equation to find decryption key d
  - $e.d=1 \mod ø(N)$ and $0 \le d \le N$
- publish their public encryption key: KU={e,N}
- keep secret private decryption key: KR={d,p,q}

RSA Use

- to encrypt a message M the sender:
  - obtains **public key** of recipient $KU=\{e,N\}$
  - computes: $C=M^e \mod N$, where $0 \leq M < N$
- to decrypt the ciphertext C the owner:
  - uses their private key $KR=\{d,p,q\}$
  - computes: $M=C^d \mod N$
- note that the message M must be smaller than the modulus N (block if needed)

Why RSA Works

- because of Euler's Theorem:
- $a^{ø(n)} \bmod N = 1$
  - where $gcd(a,N)=1$
- in RSA have:
  - $N=p.q$
  - $ø(N)=(p-1)(q-1)$
  - carefully chosen e & d to be inverses $\bmod ø(N)$
  - hence $e.d=1+k.ø(N)$ for some k
- hence :
  $C^d = (M^e)^d = M^{1+k.ø(N)} = M^1.(M^{ø(N)})^q = M^1.(1)^q = M^1 = M \bmod N$

RSA Example

1. Select primes: $p=17$ & $q=11$
2. Compute $n = pq = 17 \times 11 = 187$
3. Compute $\phi(n)=(p-1)(q-1)=16 \times 10=160$
4. Select e : $\gcd(e,160)=1$; choose $e=7$
5. Determine d: $de=1 \mod 160$ and $d < 160$
   Value is $d=23$ since $23 \times 7=161= 10 \times 160+1$
6. Publish public key $KU=\{7,187\}$
7. Keep secret private key $KR=\{23,17,11\}$

RSA Example cont

- sample RSA encryption/decryption is:
- given message $M = 88$ (nb. $88<187$)
- encryption:

  $C = 88^{7} \bmod 187 = 11$
- decryption:

  $M = 11^{23} \bmod 187 = 88$

Exponentiation

- can use the Square and Multiply Algorithm
- a fast, efficient algorithm for exponentiation
- concept is based on repeatedly squaring base
- and multiplying in the ones that are needed to compute the result
- look at binary representation of exponent
- only takes $O(\log_2 n)$ multiples for number n
  - eg. $7^5 = 7^4.7^1 = 3.7 = 10 \bmod 11$
  - eg. $3^{129} = 3^{128}.3^1 = 5.3 = 4 \bmod 11$

Exponentiation

$c \leftarrow 0;\ d \leftarrow 1$

for $i \leftarrow k$ **downto** 0

do $c \leftarrow 2 \times c$

$d \leftarrow (d \times d) \bmod n$

if $b_i = 1$

then $c \leftarrow c + 1$

$d \leftarrow (d \times a) \bmod n$

return d

RSA Key Generation

- users of RSA must:
  - determine two primes at random - p, q
  - select either e or d and compute the other
- primes p,q must not be easily derived from modulus N=p.q
  - means must be sufficiently large
  - typically guess and use probabilistic test
- exponents e, d are inverses, so use Inverse algorithm to compute the other

RSA Security

- three approaches to attacking RSA:
  - brute force key search (infeasible given size of numbers)
  - mathematical attacks (based on difficulty of computing ø(N), by factoring modulus N)
  - timing attacks (on running of decryption)

Factoring Problem

- mathematical approach takes 3 forms:
  - factor N=p.q, hence find ø(N) and then d
  - determine ø(N) directly and find d
  - find d directly
- currently believe all equivalent to factoring
  - have seen slow improvements over the years
    - as of Aug-99 best is 130 decimal digits (512) bit with GNFS
  - biggest improvement comes from improved algorithm
    - cf “Quadratic Sieve” to “Generalized Number Field Sieve”
  - barring dramatic breakthrough 1024+ bit RSA secure
    - ensure p, q of similar size and matching other constraints

Timing Attacks

- developed in mid-1990's
- exploit timing variations in operations
  - eg. multiplying by small vs large number
  - or IF's varying which instructions executed
- infer operand size based on time taken
- RSA exploits time taken in exponentiation
- countermeasures
  - use constant exponentiation time
  - add random delays
  - blind values used in calculations

Summary

- have considered:
  - principles of public-key cryptography
  - RSA algorithm, implementation, security

Cryptography · Rokib Sir

rong and provable secure ElGamal type signatures

Chapter 16.3

Santeri.Saarinen@hut.fi

Overview

- Gamal cryptosystem and El Gamal type
gnatures
- erms used
- orking reduction
- iscussion on the results
- eavy-Row reduction
- onclusion

El Gamal cryptosystem

ıblic key system based on discrete
garithm problem

ime p and primitive element α

ivate key is a and $\beta = \alpha^{a} \bmod p$

ındom number k, message x

$y_1 = \alpha^{k} \bmod p$, $y_2 = x*\beta^{k} \bmod p$

$y_2 * (y_1^{a})^{-1} \bmod p$

l Gamal example - encryption

ppose: p = 13, α = 2, a = 3,
= 2^3 mod 13 = 8,
essage x = 11, random k = 5

blic key: {p, α, β} = {13, 2, 8}

ncryption: $y_1 = 2^5$ mod 13 = 6
= 11*8^5 mod 13 = 10

phertext: (6, 10)

l Gamal example - decryption

ıblic key: {p, α, β} = {13, 2, 8}

ivate key: a = 3

phertext: y = {6, 10}

= 10 * $(6^3)^{-1}$ mod 13 = 11

El Gamal signature scheme

$g(m, k) = (y_1, y_2)$

$= \alpha^{k} \bmod p$

$= (m - a\, y_1)\, (k^{-1}) \bmod (p - 1)$

$r(m, y_1, y_2) \Leftrightarrow$

$y2 * \beta^{y1} = \alpha^{m} \bmod p$

El Gamal signature example

ıblic key: $\{p, \alpha, \beta\} = \{13, 2, 8\}$

ivate key: $a = 3$

$= 11, k = 5$

g(11, 5):

$= \alpha^{k} \bmod p = 6$

$= (m - a\, y_1)\, (k^{-1}) \bmod (p - 1) = 1$

Gamal Signature example cont.

rify:

$y2 * \beta^{y1} = \alpha^{m} \bmod p$

$* 8^{6} = 2^{11} \bmod 13$

$= 7 \Leftrightarrow$ true

gital Signature Algorithm (DSA) and
hnorr are variants of El Gamal.

Cryptography · Rokib Sir

Triplet Signature Scheme

gnature of message M is triplet (r,e,s)

s called *commitment*, committing heremal integer l. Constructed for ample: $r = g^l \bmod p$

= H(M, r), where H() is a hash function

s called signature, a linear function of l, M, H(), signing key)

Secure Signature Scheme

gnature scheme is denoted by (*Gen*, *Sign*, *erify*)

en generates private and public key

gn signs message and *Verify* verifies

gnature scheme is (*t(k)*, *Adv(k)*), if there ists no forger able to forge a signature for sufficiently large k.

Cryptography · Rokib Sir

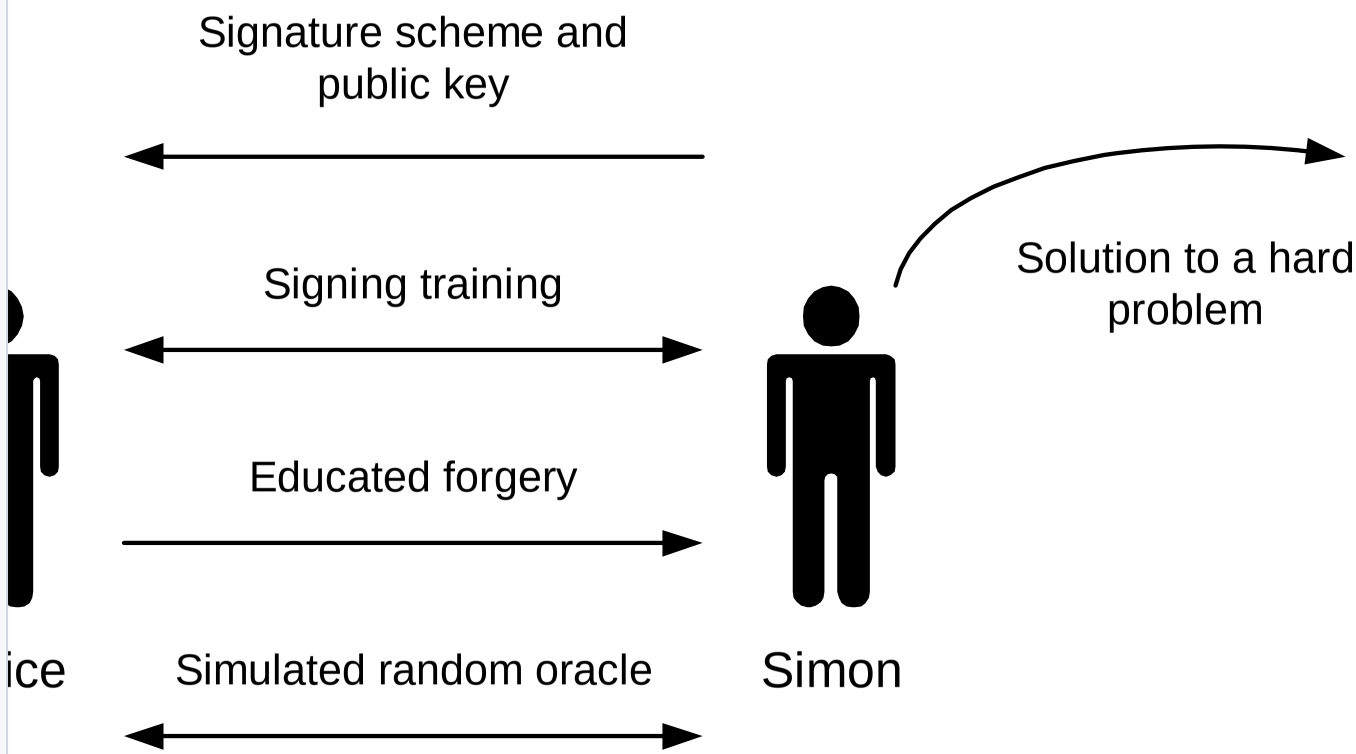
Reduction

ansformation from *t(k)*, *Adv(k)* to *t'(k)*, *lv'(k)*, which is corresponding solution to hard problem (e.g. discrete logarithm)

ain aim to make solution to a hard oblem "too easy".

milarity between between the two efforts pends on the efficency of the reduction

Setup


Signature scheme and public key
Signing training
Educated forgery
Simulated random oracle
Solution to a hard problem
ice
Simon

Cryptography · Rokib Sir

Non-adaptive attack

triplet version of El Gamal is used

o signing training needed

mon operates as simulated random oracle

r H() queries

First lot of runs

- *Adv(k)* runs needed by Malice
- mon maintains list of e = H(M, r) livered to Malice
- hen Malice outputs a forgery, he has eried the corresponding e

Cryptography · Rokib Sir

Second lot of runs

alice re-runs *1/Adv(k)* times

mon resets his RO-answers

ecause on birthday-paradox, two signature
irs (M, (r,e,s)) and (M', (r',e',s')) satisfy
M,r) = (M',r') after number of tries

Cryptography · Rokib Sir

xtraction of discrete logarithm

$^{r}r^{s} = g^{e} \pmod p$, $y^{r}r^{s'} = g^{e'} \pmod p$

$r + ls = e \pmod q$, $xr + ls' = e' \pmod q$

$= (e - e')/(s - s') \pmod q$

$= (e - ls)/r \pmod q$

mon does not care of Malice's method,
t is able to extract discrete logarithm.

Cryptography · Rokib Sir

Reduction results

mon's advantage $Adv'(k) = 1 / (q_h^{0,5})$

mon's time cost $t' = 2(t+q_h)/Adv(k)$

s the time Malice needs for a forgery

iis works only if Malice does not care he fooled

laptive chosen-message attack

mon simulates also signing of the essages, but does not posses the signing y. Though signature can be verified!

r signing query, Simon returns:

$= g^u y^v \pmod p$

$= -rv^{-1} \pmod{p - 1}$

$= -ruv^{-1} \pmod{p - 1}$

where u and v are random integers

Cryptography · Rokib Sir

ACM-attack results

$= 2 * (t(k) + q_H*\tau) + OB(q_s*k^3)/Adv(k)$

$'(k) = q_H^{-0,5}$

is number of signing queries

is number of hash-queries

s time consumed in answering a query

Discussion

- ıe proof suggests that vulnerable parts of
  ıs kind of signature are discrete logarithm
  d the hash-function
- eduction should run $q_H^{0,5}$ times, which
  akes the total time $O(q_H^{3/2} / Adv)$
- ao suggests 2^{50} hash queries
  - $O(2^{75} / Adv)$

Heavy-Row technique

reated for zero-knowledge identification,
t applies to El Gamal also

atrix-based approach featuring Malice
d Simon

vo forged signatures lead to contradiction
in forking technique.

Conclusion

is possible to reduct from forgering a plet El Gamal signature to solving screte logarithm in constant time.

sing Simon the Simulator offers a good ol to build the reduction.

ElGamal Public Key Cryptography

CS 303 Alg. Number Theory & Cryptography

Jeremy Johnson

Taher ElGamal, "A Public-Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms", IEEE Transactions on Information Theory, v. IT-31, n. 4, 1985, pp469–472 or CRYPTO 84, pp10–18, Springer-Verlag.

Outline

- Primitive Element Theorem
- Diffie Hellman Key Distribution
- ElGamal Encryption
- ElGamal Digital Signatures

Public Key Cryptography

- Let M be a message and let C be the encrypted message (ciphertext). A public key cryptosystem has a separate method E() for encrypting and D() decrypting.
  - D(E(M)) = M
  - Both E() and D() are easy to compute
  - Publicly revealing E() does not make it easy to determine D()
  - E(D(M)) = M - needed for signatures

- The collection of E()'s are made publicly available but the D()'s remain secret. Called a one-way trap-door function (hard to invert, but easy if you have the secret information)

Order

- Definition. Let $b \in Z_n^*$ The order of b is the smallest positive integer satisfying $b^e \equiv 1 \pmod{n}$.

- Theorem 1. If b has order e modulo n and if j is a positive integer such that $b^j \equiv 1 \pmod{n}$, then $e|j$.

Proof. $j = q \times e + r$, $0 \leq r < e$.

$b^j \equiv 1 \equiv (b^e)^q \, b^r \equiv b^r \pmod{n}$.

This implies that r = 0, since e is the smallest power of b equivalent to 1 mod n.

- Corollary 2. Let $b \in Z_n^*$. $ord(b)|\phi(n)$.

Primitive Element Theorem

- $Z_p^* = \langle \alpha \rangle$, i.e. $\text{ord}(\alpha) = p-1$.

- Example
  - $Z_7^* = \langle 3 \rangle$ $3^1=3, 3^2=2, 3^3=6, 3^4=4, 3^5=5, 3^6=1$
  - $Z_{13}^* = \langle 2 \rangle$ $2^1=2, 2^2=4, 2^3=8, 2^4=3, 2^5=6, 2^6=12, 2^7=11, 2^8=9, 2^9=5, 2^{10}=10, 2^{11}=7, 2^{12}=1$

- Note. $\text{ord}(\alpha) = p-1 \Rightarrow \{1, \alpha, \alpha^2, \ldots, \alpha^{p-1}\}$ distinct.

Discrete Logarithms

- Discrete log problem
  - Given $Z_p^* = <\alpha>$
  - $\log_\alpha(y) = x$, if $y = \alpha^x$.

- Example
  - $Z_{13}^* = <2>$ $2^1=2$, $2^2=4$, $2^3=8$, $2^4=3$, $2^5=6$, $2^6=12$, $2^7=11$, $2^8=9$, $2^9=5$, $2^{10}=10$, $2^{11}=7$, $2^{12}=1$
  - $\text{Log}_2(5) = 9$.

Properties of Primitive Elements

- Theorem 3. If b has order e modulo n, then ord(b^i) = e/gcd(e,i).
- Theorem 4. Let p be a prime and d a divisor of p-1, then the number of positive integers less than p with order d is $\phi(d)$.
- Corollary 5. The number of primitive elements mod p is equal to $\phi(p-1) > 1$.

Some Lemmas

- Lemma 6. Let P(x) be a polynomial of degree t and let p be a prime. If p does not divide the coefficient of x^t in P(x), then $P(x) \equiv 0 \pmod{p}$, has at most t solutions mod p.

Proof. By induction on the degree of P(x)=t.

$P(x_1) = 0 \Rightarrow P(x) = P_1(x) \times (x - x_1)$, and the degree of $P_1(x)$ = t-1.

- Lemma 7. The sum of $\phi(d)$ over the divisors of n = n.
  - Example: n=12. $\phi(1)+ \phi(2)+ \phi(3)+ \phi(4)+ \phi(6)+ \phi(12)=1+1+2+2+2+4 = 12$.

Primitive Element Theorem

- Theorem. Let p be a prime and d a divisor of p-1, then the number of positive integers less than p with order d is $\phi(d)$.

Proof. If there is an element a of order d, then by Theorem 3, a^i, gcd(i,d)=1 is also of order d. By Lemma 6, 1, a, a^2,…,a^{d-1} are the roots of $P(x)=x^d-1$, and there $\phi(d)$ elements of order d. Since every elements is of order d|p-1 and $p-1 = \Sigma_{d|p-1} \phi(d)$, there must be an element of order d for every d|p-1 and hence exactly $\phi(d)$ of them.

Public Key Distribution

- The goal is for two users to securely exchange a key over an insecure channel. The key is then used in a normal cryptosystem

- Diffie-Hellman Key Exchange
  - $A = g^a \bmod p$ (p prime, g primitive – all elements of $(Z_p)^*$are powers of g) [Alice sends A to Bob]
  - $a = \log_g A \bmod p$ [discrete log]
  - $B = g^b \bmod p$ [Bob sends B to Alice]
  - $K = g^{ab} \bmod p$ [shared key]
  - $A^b = g^{ab} = B^a \bmod p$

ElGamal Encryption

- $Z_p^* = \langle g \rangle$, $m \in Z_p$ message
  - B encrypts a message to A.
- Alice: a random, $h = g^a$, public key = (p, g,A)
- Bob: k random (ephemeral key), $c_1 = g^k$, shared key $K = A^k = g^{ak}$
  - $E_A(m) = (c_1,c_2)$, $c_2 = mK \bmod p$.
  - $D_A((c1,c2)) = c_2 * (1/K) \bmod p$, $K = c_1{}^a = g^{ak}$
- Security depends on Computational Diffie-Hellman (CDH) assumption: given (g, g^a, g^b) it is hard to compute g^{ab}
- Do not use same k twice

ElGamal Digital Signature

- $Z_p^* = \langle g \rangle$, $m \in Z_p$ message
  - A signs message m.

- Alice: $A = g^a$, public key = (p, g,A), secret key = x.
- Alice: k random with gcd(k,p-1)=1
  - $r = g^k \pmod p$
  - $s = (m - xr)(1/k) \bmod p-1$ $[m = sk + xr \pmod{p-1}]$
  - Signature = (r,s)
  - Verify $g^m = r^s h^r$

The Paillier Cryptosystem

A Look Into The Cryptosystem And Its Potential Application

By

Michael O'Keeffe

The College of New Jersey

Mathematics Department

April 18, 2008

ABSTRACT

So long as there are secrets, there is a need for encryption to help guard these secrets. The Paillier Cryptosystem is an encryption scheme that can be used to conceal information, with a few interesting properties. These properties, when creatively applied, allow the Paillier Cryptosystem to be used in ways that other cryptographic systems simply can't be used. This paper will explore how the Paillier Cryptosystem works, how these properties arise, and one way in which the system can be used in a real world situation as a result of these properties.

1. INTRODUCTION

1.1 A Brief History of Cryptography

There are simply times when the intended recipient of a message needs to be the only person able to gather the information contained inside it. To this end, there exists the need for the ability to hide the contents of a message from all but the intended recipient. Primitively, one may choose to physically conceal a message in a location of which only the recipient is aware. However, there exists the risk of the message's discovery, at which point all information contained within the message is no longer secure. Thus, the need for encryption arises: a way in which to alter a message in such a way that, should the message be intercepted by someone other than the intended recipient, it would be difficult, if not practically impossible, for this person to absorb the information contained within the message. However, this process of concealment must be reversible, as the intended recipient needs to be able to undo this process of encryption, in order to be able to gather the information intended to be conveyed.

From humble beginnings of simply "shifting" letters down the alphabet (if you intend to write an 'a', write 'b' instead; 'b' becomes 'c', etc.), mathematics was already involved in encryption – even if people weren't strictly aware of the math they were performing (the addition of a constant, modulo 26). So-called "shift-ciphers", and other means of encryption

Cryptography · Rokib Sir

in which the decryption, or undoing of the encryption process, requires vital information about how the message was encrypted to begin with, are called private key cryptosystems. The problem with private key systems is that the private key must be given to everyone who wishes to either encrypt or decrypt a message. With multiple copies of the key floating around, the security of a private key system is highly contingent upon the trust placed in each individual key holder. There is also the problem of how to securely give the key to someone to begin with, aside from a face-to-face encounter (which isn't always practical). To meet this problem, so-called "public key" cryptosystems were invented. Public key systems publish to one and all a means of encrypting messages, while holding private a single means of decrypting any of these messages. The strength of a public key cryptosystem lies in the difficulty of determining a means of decryption given only a means of encryption.

1.2 The Paillier Cryptosystem

The Paillier Cryptosystem is a modular, public key encryption scheme, created by Pascal Paillier, with several interesting properties. This paper will explore Paillier's work [3], beginning by showing how to encrypt and decrypt messages using this cryptosystem, with the underlying mathematical principles that make the system work clearly outlined. It is assumed that the reader is familiar, to some degree, with modular arithmetic, as well as the concept of converting an alphanumeric message into a purely numeric message, which can be broken into blocks, m_i, such that, for each i, $0 < m_i < n$, for a predetermined value, n. Also, the term plaintext will be used to refer to a message that is numeric, but is not encrypted, while the term cipher text will be used to refer to plaintexts which have been encrypted, but not yet decrypted. Following an examination of the encryption and decryption process, several of the aforementioned interesting properties will be listed, and the math behind them explored. One property in particular, the addition of plaintexts through multiplication of cipher texts, will then be looked at in terms of its potential application to a form of electronic voting, in order to illustrate the system's potential.

2. USING THE PAILLIER CRYPTOSYSTEM

2.1 Encryption

In order to encrypt a message using the Paillier cryptosystem, a public key must first be established.

To construct the public key, one must choose two large primes, p and q, then calculate their product, $n = p \cdot q$. Then a semi-random, nonzero integer, g, in $\mathbb{Z}_{n^2}$, must be selected, such that the order of g is a multiple of n in $\mathbb{Z}^*_{n^2}$ [$\mathbb{Z}^*_{n^2}$ being the units, or invertible elements, of $\mathbb{Z}_{n^2}$]. It is said that g is semi-random since there are a few values which will not work, for reasons which will be addressed in the *Decryption* section of this paper. For ease of calculations, the examples in this paper will choose small primes, to create a small n. Let $p = 7$ and $q = 11$, then $n = p \cdot q = 7 \cdot 11 = 77$. Next, an integer g must be selected from $\mathbb{Z}^*_{n^2}$, such that the order of g is a multiple of n in $\mathbb{Z}^*_{n^2}$. The integer g must also satisfy another property, which will be discussed in *§2.2 Decryption*. If $g = 5652$ (randomly chosen by *Mathematica*), then all necessary properties, including the yet to be specified condition, are met, as $|g| = 2310 = 30 \cdot 77$ in $\mathbb{Z}^*_{n^2}$. Thus, the public key for the examples in this paper will be $(n, g) = (77, 5652)$.

With the public key now established, anyone can encrypt a message to send to the holder of the private key, as shown in Figure 1.

Example	General
Let $m = 42$	Create a message, m, with $m \in \mathbb{Z}_n$
Let $r = 23$	Choose a random, nonzero integer, $r \in \mathbb{Z}^*_n$
$c \equiv (5652)^{42} \cdot (23)^{77} \bmod 5929$ $\equiv (4019)(606) \equiv 4624 \bmod 5929$	Compute $c \equiv g^m \cdot r^n \bmod n^2$

Figure 1: Encryption using the Paillier Cryptosystem, given $n = 77$, $g = 5652$.

The encrypted message is then just c. The holder of the private key does not need to know the value of r in order to decrypt c.

2.2 Decryption

Given an encrypted message, c, and knowing the values p, q and g, one can decrypt c. Note that Carmichael's function, $\lambda(n) = \text{lcm}[(p - 1)(q - 1)]$, is easily computable given the values of p and q. Also note that, given a $g \in \mathbb{Z}^*_{n^2}$, such as the g chosen for this public key, Carmichael's Theorem guarantees that $g^{\lambda(n)} \equiv 1 \bmod n$. Carmichael's Theorem states that if two integers, a and n, are relatively prime, then $a^{\lambda(n)} \equiv 1 \bmod n$. Since g is a unit modulo n^2,

it is relatively prime to n^2, which means it's relatively prime to n, thus Carmichael's Theorem applies. For any decryption with the public key (n, g), regardless of the value of c, the calculation of $g^{\lambda(n)} \bmod n^2$ is necessary. This resulting value, an element of $\mathbb{Z}^*_{n^2}$, will, by Carmichael's Theorem [1], be congruent to 1 mod n. Thus, subtracting one from this resulting value will give a number that is divisible by n (congruent to zero mod n). So, compute $g^{\lambda(n)} \bmod n^2$, subtract one from this value, then divide that number by n, as depicted in Figure 2.

Example	General
$\lambda(77) = \text{lcm}(6, 10) = 30$ $L(5652^{30} \bmod 5929) = L(3928)$ $L(3928) = (3928 - 1)/77 = 3927/77 = 51$	Define $L(u) = (u - 1)/n$ Compute $L(g^{\lambda(n)} \bmod n^2) = k$

Figure 2: Computation of $L(g^{\lambda(n)} \bmod n^2)$, given $n^2 = 5929$, $g = 5652$.

Notice that since $g^{\lambda(n)}$ is being calculated mod n^2, it can be viewed as a number greater than or equal to zero, but strictly less than n^2, so dividing this number by n results in a value, k, greater than or equal to zero, but strictly less than n: $k \in \mathbb{Z}_n$. Since $n = p \cdot q$, so long as k is not congruent to a multiple of p or q mod n, then k has an inverse, so $k \in \mathbb{Z}^*_n$. This is the previously undefined property that must be satisfied by g, which was mentioned in *§2.1 Encryption*. Values of g such that $L(g^{\lambda(n)} \bmod n^2)$ is congruent to a multiple of p or q mod n are the few exceptions of semi-random g values with orders divisible by n that must be excluded. If such a value is chosen, simply pick another value for g, and check that this property holds before publishing the public key. So, assuming k is not congruent to p or q mod n, then k has an inverse mod n, so compute $\mu \equiv k^{-1} \bmod n$, as in Figure 3. For any decryption involving the public key (n, g), the value of μ will always be the same, and will always be necessary.

Example	General
$\mu \equiv 51^{-1} \equiv 74 \bmod 77$	Compute $\mu \equiv k^{-1} \bmod n$

Figure 3: μ, the inverse of $L(g^{\lambda(n)} \bmod n^2)$ in $\mathbb{Z}^*_n$ is necessary for every decryption. Here n = 77, and $L(g^{\lambda(n)} \bmod n^2) = 51$

To decrypt c, one must calculate $m \equiv L(c^{\lambda(n)} \bmod n^2) \cdot \mu \bmod n$, as shown in Figure 4.

Example	General
$m \equiv L(4624^{30} \equiv 4852 \bmod 5929) \cdot 74 \bmod 77$ $m \equiv 63 \cdot 74 \equiv 4662 \equiv 42 \bmod 77$	$m \equiv L(c^{\lambda(n)} \bmod n^2) \cdot \mu \bmod n$
Figure 4: Decryption in the Paillier Cryptosystem. Here, $n = 77$, $c \equiv 4624 \bmod n$, and $\mu \equiv 74 \bmod n$	

2.3 Mathematics in the Cryptosystem

In order to understand why this decryption process works, we must first introduce the following function:

$$\varepsilon_g: \mathbb{Z}_n \times \mathbb{Z}^*_n \rightarrow \mathbb{Z}^*_{n^2}$$
$$\varepsilon_g(x, y) \rightarrow g^x \cdot y^n \bmod n^2$$

Notice: The output of $\varepsilon_g(m, r)$ is the encryption of m, using the random integer r. Recall that, when choosing g, there was the requirement that the order of g mod n^2 be a multiple of n. This is necessary since, if the order of g is a nonzero multiple of n, then ε_g is bijective. To prove this theorem, the following lemma [2] must first be established:

Lemma: Given any two finite sets, A and B, with $|A| = |B| = s$, a function *f*: $A \rightarrow B$ is injective iff it is surjective.

Proof of Lemma: Let $A = \{a_1, a_2, \ldots, a_s\}$, $B = \{b_1, b_2, \ldots, b_s\}$

Assume *f* is injective, then $f(a_1), f(a_2), \ldots f(a_s)$ are s *distinct* elements in B. Since B only contains s elements, these must be all s elements of B in some order. Therefore every element of B is reachable through *f*, and *f* is surjective.

Assume *f* is surjective, then every $b_i \in B$ is reachable through $f(a_j)$ for some $a_j \in A$. Each element a_j maps to only one element in B. Since all s elements in B are reachable through *f*, there are at least s distinct elements in A which map, through *f*, to each of these s distinct elements in B. Since $|A| = s$, these s distinct elements are all the elements of A, so each element of A maps to a distinct element in B, making *f* injective. ■

With this lemma in mind, the above theorem can be proven.

Theorem: If the order of g is a nonzero multiple of n, then ε_g is bijective for $\varepsilon_g(x, y) \equiv g^x \cdot y^n \bmod n^2$.

Proof of Theorem: Assume the order of g is a nonzero multiple of n.

First, notice the order of $\mathbb{Z}^*_{n^2} = \phi(n^2) = n \cdot \phi(n) = |\ \mathbb{Z}_n \times \mathbb{Z}^*_n\ |$, so the sets contain the same number of elements. Therefore, if ε_g is injective, it is also surjective by the above lemma, and is thus bijective. So, it will suffice to show ε_g is injective to show it is bijective.

Assume $g^{X_1} \cdot y_1^n \equiv g^{X_2} \cdot y_2^n \bmod n^2$. It follows that $g^{X_1 - X_2} \cdot (y_1/y_2)^n \equiv 1 \bmod n^2$. Raising both sides to $\lambda(n)$, we get that $g^{\lambda(n) \cdot (X_1 - X_2)} \cdot (y_1/y_2)^{n \cdot \lambda(n)} \equiv 1 \bmod n^2$. Recall that Carmichael's Theorem told us that an element of $\mathbb{Z}^*_{n^2}$, raised to the $\lambda(n)$ power, will be congruent to 1 mod n. The same theorem also promises that an element of $\mathbb{Z}^*_{n^2}$, raised to the $n \cdot \lambda(n)$ power, will be congruent to 1 mod n^2[1]. Since y_1 and y_2^{-1} are elements of $\mathbb{Z}^*_{n^2}$, their product, y_1/y_2, is also an element of $\mathbb{Z}^*_{n^2}$, which means $(y_1/y_2)^{n \cdot \lambda(n)} \equiv 1 \bmod n^2$, so then $g^{\lambda(n) \cdot (X_1 - X_2)} \cdot (y_1/y_2)^{n \cdot \lambda(n)} \equiv g^{\lambda(n) \cdot (X_1 - X_2)} \equiv 1 \bmod n^2$.

This implies that $\lambda(n) \cdot (x_1 - x_2)$ is a multiple of the order of g. Since we're assuming the order of g is a nonzero multiple of n, this makes $\lambda(n) \cdot (x_1 - x_2)$ a multiple of n. Since n divides $\lambda(n) \cdot (x_1 - x_2)$, and GCD($\lambda(n)$, n) = 1, it follows that $n \mid x_1 - x_2$, or $x_1 - x_2 \equiv 0 \bmod n \Rightarrow x_1 \equiv x_2 \bmod n$. Since x_1 and x_2 are elements mapped from $\mathbb{Z}_n$, their congruence mod n ensures their equality in $\mathbb{Z}$.

Going back to the equation $g^{X_1 - X_2} \cdot (y_1/y_2)^n \equiv 1 \bmod n^2$, with $x_1 = x_2$, we get that $(y_1/y_2)^n \equiv 1 \bmod n^2 \Rightarrow y_1^n \equiv y_2^n \bmod n^2$. This is satisfied when $y_1 \equiv y_2 \bmod n$.

Consider $y_1 \equiv y_2 + \alpha \cdot n \bmod n^2 \Rightarrow y_1 \equiv y_2 + \alpha \cdot n \equiv y_2 \bmod n$

$y_1^n \equiv (y_2 + \alpha \cdot n)^n \equiv y_2^n + n \cdot y_2^{n-1} \cdot (\alpha \cdot n) + [\text{higher powers of n}] \bmod n^2$

$\equiv y_2^n + y_2^{n-1} \cdot \alpha \cdot n^2 \equiv y_2^n \bmod n^2$

---

[1] For primes, p_1, p_2: $\lambda(p_1^{k_1}) = p_1^{(k_1-1)} \cdot (p_1 - 1)$ and $\lambda(p_1^{k_1} \cdot p_2^{k_2}) = \text{lcm}(\lambda(p_1^{k_1}), \lambda(p_2^{k_2}))$. So $\lambda(n^2) = \lambda(p^2 \cdot q^2) = \text{lcm}(\lambda(p^2), \lambda(q^2))$ $\lambda(n^2) = \text{lcm}(p \cdot (p-1), q \cdot (q-1)) = p \cdot q \cdot \text{lcm}(p-1, q-1) = p \cdot q \cdot \lambda(n)$. So $\lambda(n^2) = n \cdot \lambda(n)$.

Therefore $x_1 = x_2$, and since $y_1, y_2 \in \mathbb{Z}^*_n$, $y_1 = y_2$ ■

This means that, given any $w \in \mathbb{Z}^*_{n^2}$, with n fixed, for a chosen $g \in \mathbb{Z}^*_{n^2}$, whose order is a nonzero multiple of n, the pair (x, y) such that $\varepsilon_g(x, y) \equiv w \bmod n^2$ is a unique pair. For ease of notation, given $\varepsilon_g(x, y) \equiv g^x \cdot y^n \equiv w \bmod n^2$, let $[\![w]\!]_g = x$. That is, with n fixed, and a proper g specified, let $[\![w]\!]_g$ be the unique element, x, from $\mathbb{Z}_n$, such that $\varepsilon_g(x, y) \equiv w \bmod n^2$.

Notice that ε_g maps to all elements of $\mathbb{Z}^*_{n^2}$, and that g itself is an element of $\mathbb{Z}^*_{n^2}$. Thus, for another element, t, of $\mathbb{Z}^*_{n^2}$, such that t has an order that is a nonzero multiple of n, one could compute $[\![g]\!]_t$ for the g chosen for this particular cryptosystem key.

Powers of (1+n) in $\mathbb{Z}_{n^2}$	
Consider $(1+n) \in \mathbb{Z}^*_{n^2}$:	$(1+n)^2 \equiv 1 + 2n + n^2 \equiv 1 + 2n \bmod n^2$.
	$(1+n)^3 \equiv 1 + 3n + 3n^2 + n^3 \equiv 1 + 3n \bmod n^2$
	$(1+n)^v \equiv 1 + v \cdot n +$ [higher powers of n] $\equiv 1 + v \cdot n \bmod n^2$

Figure 5: Powers of (1+n) can be reduced to 1 + (n times the given power) when working mod n^2

As established by Figure 5, $(1+n)^n \equiv 1 + n \cdot n \equiv 1 \bmod n^2$. Clearly n, itself a multiple of n, is the order of (1+n) in $\mathbb{Z}^*_{n^2}$ (any lower, nonzero, positive power, v, would be congruent to $1 + v \cdot n$, which is not congruent to 1 mod n^2), with $(1+n)^{(n-1)}$ being its inverse (thus showing it clearly belongs in $\mathbb{Z}^*_{n^2}$ to begin with). (1+n) then meets the properties desired in the aforementioned "other" element, t, and we can compute $[\![g]\!]_{(1+n)}$. That is to say, g can be expressed as $g \equiv \varepsilon_{(1+n)}(t, z) \equiv (1+n)^t \cdot z^n \bmod n^2$, for the unique pair (t, z), where $t = [\![g]\!]_{(1+n)}$.

Recall that, when encrypting a message m, one computes $c \equiv \varepsilon_g(m, r) \equiv g^m \cdot r^n \bmod n^2$. But, we just learned that g can be expressed as $g \equiv \varepsilon_{(1+n)}([\![g]\!]_{(1+n)}, z) \equiv (1+n)^{[\![g]\!]_{(1+n)}} \cdot z^n \bmod n^2$. Substituting, we get the following:

$$\begin{aligned} c &\equiv g^m \cdot r^n \equiv [(1+n)^{[\![g]\!]_{(1+n)}} \cdot z^n]^m \cdot r^n \bmod n^2 \\ &\equiv (1+n)^{m \cdot [\![g]\!]_{(1+n)}} \cdot z^{m \cdot n} \cdot r^n \bmod n^2 \\ &\equiv (1+n)^{m \cdot [\![g]\!]_{(1+n)}} \cdot (z^m \cdot r)^n \bmod n^2 \end{aligned}$$

Note: $z \in \mathbb{Z}^*_n \Rightarrow z^m \in \mathbb{Z}^*_n$, and $r \in \mathbb{Z}^*_n$, therefore $z^m \cdot r \in \mathbb{Z}^*_n$

$$\text{So, } c \equiv \varepsilon_{(1+n)}(m \cdot [\![g]\!]_{(1+n)}, z^m \cdot r) \bmod n^2$$

Then, by definition, $[\![c]\!]_{(1+n)} \equiv m \cdot [\![g]\!]_{(1+n)} \Rightarrow m \equiv [\![c]\!]_{(1+n)} \cdot \{[\![g]\!]_{(1+n)}\}^{-1} \bmod n$

Notice: $[\![c]\!]_{(1+n)}$ is defined as the element from $\mathbb{Z}_n$ that maps to the power of (1+n) under $\varepsilon_{(1+n)}$ to yield c, which is why the above product is computed mod n, since all elements are in $\mathbb{Z}_n$.

Thus, if we can calculate the inverse of $[\![g]\!]_{(1+n)}$, which will always be the same regardless of the encrypted value of c, then decryption of any message would involve determining $[\![c]\!]_{(1+n)}$, multiplying it by this predetermined, set value of $[\![g]\!]_{(1+n)}$, and calculating the result mod n.

Recall, in the decryption process, we calculated $\mu \equiv L(g^{\lambda(n)} \bmod n^2)^{-1} \bmod n$, claiming it was a predetermined, set value that would be necessary for decryption, regardless of the value of m, c, or r. We will now see that $\mu \equiv L(g^{\lambda(n)} \bmod n^2)^{-1} \equiv \{\lambda(n)\cdot[\![g]\!]_{(1+n)}\}^{-1} \bmod n$, after which we will see that the calculation of $L(c^{\lambda(n)} \bmod n^2)$ yields $\lambda(n)\cdot[\![c]\!]_{(1+n)} \bmod n$, thus making its product with μ congruent to $\lambda(n)\cdot[\![c]\!]_{(1+n)}\cdot\lambda(n)^{-1}\cdot[\![g]\!]_{(1+n)}^{-1} \equiv [\![c]\!]_{(1+n)}\cdot[\![g]\!]_{(1+n)}^{-1} \equiv m \bmod n$.

Consider $g^{\lambda(n)} \bmod n^2$:

$$g^{\lambda(n)} \equiv [(1+n)^{[\![g]\!](1+n)}\cdot z^n]^{\lambda(n)} \equiv (1+n)^{\lambda(n)\cdot[\![g]\!](1+n)}\cdot z^{n\cdot\lambda(n)} \bmod n^2$$

Carmichael's Theorem again tells us that $z^{n\cdot\lambda(n)} \equiv 1 \bmod n^2$ since z is an element of $\mathbb{Z}^*_{n^2}$.

Thus:

$$\begin{aligned} g^{\lambda(n)} &\equiv (1+n)^{\lambda(n)\cdot[\![g]\!](1+n)} \bmod n^2 \\ &\equiv 1 + \lambda(n)\cdot[\![g]\!]_{(1+n)}\cdot n + [\text{higher powers of } n] \bmod n^2 \\ &\equiv 1 + \lambda(n)\cdot[\![g]\!]_{(1+n)}\cdot n \bmod n^2 \end{aligned}$$

Now, we apply the function L(u) to $g^{\lambda(n)} \bmod n^2$, which is defined as L(u) = (u–1)/n:

$$\begin{aligned} L(g^{\lambda(n)} \bmod n^2) &\equiv L(1 + \lambda(n)\cdot[\![g]\!]_{(1+n)}\cdot n) \bmod n \\ &\equiv \{(1 + \lambda(n)\cdot[\![g]\!]_{(1+n)}\cdot n)-1\}/n \bmod n \\ &\equiv \{\lambda(n)\cdot[\![g]\!]_{(1+n)}\cdot n\}/n \bmod n \\ &\equiv \lambda(n)\cdot[\![g]\!]_{(1+n)} \bmod n \end{aligned}$$

So, $L(g^{\lambda(n)} \bmod n^2) \equiv \lambda(n)\cdot[\![g]\!]_{(1+n)} \bmod n$, and $\mu \equiv L(g^{\lambda(n)} \bmod n^2)^{-1} \equiv \{\lambda(n)\cdot[\![g]\!]_{(1+n)}\}^{-1} \bmod n$, just as claimed.

Similarly, Carmichael's Theorem will help us reduce $L(c^{\lambda(n)} \bmod n^2)$ in terms of $\varepsilon_{(1+n)}$:

Consider $c^{\lambda(n)} \bmod n^2$:

$$\begin{aligned} c^{\lambda(n)} &\equiv [(1+n)^{[\![c]\!](1+n)} \cdot d^n]^{\lambda(n)} \bmod n^2 \\ &\equiv (1+n)^{\lambda(n)\cdot[\![c]\!](1+n)} \cdot d^{\,n\cdot\lambda(n)} \bmod n^2 \\ &\equiv (1+n)^{\lambda(n)\cdot[\![c]\!](1+n)} \bmod n^2 \text{ (by Carmichael's Theorem)} \\ &\equiv 1 + \lambda(n)\cdot[\![c]\!]_{(1+n)}\cdot n + [\text{higher powers of } n] \bmod n^2 \\ \therefore\ c^{\lambda(n)} &\equiv 1 + \lambda(n)\cdot[\![c]\!]_{(1+n)}\cdot n \bmod n^2 \end{aligned}$$

$$\begin{aligned} \text{So } L(c^{\lambda(n)} \bmod n^2) &\equiv L(1 + \lambda(n)\cdot[\![c]\!]_{(1+n)}\cdot n) \\ &\equiv \{(1 + \lambda(n)\cdot[\![c]\!]_{(1+n)}\cdot n) - 1\}/n \bmod n \\ &\equiv \{\lambda(n)\cdot[\![c]\!]_{(1+n)}\cdot n\}/n \\ &\equiv \lambda(n)\cdot[\![c]\!]_{(1+n)} \bmod n \end{aligned}$$

So, $\mu \equiv L(g^{\lambda(n)} \bmod n^2)^{-1} \equiv \{\lambda(n)\cdot[\![g]\!]_{(1+n)}\}^{-1} \bmod n$, and $L(c^{\lambda(n)} \bmod n^2) \equiv \lambda(n)\cdot[\![c]\!]_{(1+n)} \bmod n$,
Therefore $L(c^{\lambda(n)} \bmod n^2)\cdot\mu \equiv \lambda(n)\cdot[\![c]\!]_{(1+n)}\cdot\lambda(n)^{-1}\cdot[\![g]\!]_{(1+n)}^{-1} \equiv [\![c]\!]_{(1+n)}\cdot[\![g]\!]_{(1+n)}^{-1} \equiv m \bmod n$

Notice that, for a given public key choice of (n, g), μ is always the same, and needs to only be calculated once in order to decrypt an infinite number of messages. This means decryption involves one exponentiation mod n^2, the simple calculation of L(u), then a simple multiplication mod n, making decryption a relatively simple process, whose biggest complexity is a single exponentiation mod n^2.

3. Properties Of The Paillier Cryptosystem

The Paillier Cryptosystem takes full advantage of Carmichael's Theorem, as evident in the following properties (note: D(u) means decrypting u, E(u) means encrypting u):

Property 1: Multiplying encrypted messages results in the addition of the original plaintexts mod n.

Formally: $D[E(m_1)\cdot E(m_2) \bmod n^2] \equiv m_1 + m_2 \bmod n$

Proof: Let $c_1 \equiv g^{m_1}\cdot r_1^{\,n} \bmod n^2$, and $c_2 \equiv g^{m_2}\cdot r_2^{\,n} \bmod n^2$

Then $c_1\cdot c_2 \equiv g^{m_1}\cdot r_1^{\,n}\cdot g^{m_2}\cdot r_2^{\,n} \bmod n^2 \Rightarrow c_1\cdot c_2 \equiv g^{m_1}\cdot g^{m_2}\cdot r_1^{\,n}\cdot r_2^{\,n} \equiv g^{m_1+m_2}\cdot(r_1\cdot r_2)^n \bmod n^2$

Notice: $r_1, r_2 \in \mathbb{Z}^*_n$, thus $r_1 \cdot r_2 \in \mathbb{Z}^*_n$, so this is the encryption of a new message, $m_1 + m_2$, with the random element $r_1 \cdot r_2$. Decrypting the product of c_1 and c_2 begins by raising $(c_1 \cdot c_2)$ to the $\lambda(n)$:

$(c_1 \cdot c_2)^{\lambda(n)} \equiv g^{\lambda(n)\cdot(m_1+m_2)} \cdot (r_1 \cdot r_2)^{n \cdot \lambda(n)} \equiv g^{\lambda(n)\cdot(m_1+m_2)} \bmod n^2$ by Carmichael's Theorem

It's still true that $g^{\lambda(n)} \equiv (1+n)^{\lambda(n)\cdot[\![g]\!](1+n)} \bmod n^2$, so substituting, we have:

$(c_1 \cdot c_2)^{\lambda(n)} \equiv (1+n)^{\lambda(n)\cdot[\![g]\!](1+n)\cdot(m_1+m_2)} \equiv 1 + (m_1+m_2)\cdot\lambda(n)\cdot[\![g]\!]_{(1+n)}\cdot n \bmod n^2$

So, $L((c_1 \cdot c_2)^{\lambda(n)} \bmod n^2) \equiv (m_1+m_2)\cdot\lambda(n)\cdot[\![g]\!]_{(1+n)} \bmod n$, and μ is still the inverse of $\{\lambda(n)\cdot[\![g]\!]_{(1+n)}\}$, so completing the decryption process by multiplying by μ, the result is $m_1 + m_2 \bmod n$.

Example: Assume the public key $(n, g) = (77, 5652)$ has been published. If one were to encrypt two messages, say $(m_1, r_1) = (42, 23)$ and $(m_2, r_2) = (15, 61)$, the resulting cipher texts would be $E(m_1) = 4624$ and $E(m_2) = 1306$.

The product of these cipher texts, $E(m_1)\cdot E(m_2)$, is

$$E(m_1)\cdot E(m_2) \equiv 4624 \cdot 1306 \equiv 6038944 \equiv 3222 \bmod 5929.$$

The decryption process begins by calculating $L([E(m_1)\cdot E(m_2)]^{\lambda(n)} \bmod n^2)$:

$$L(3222^{30} \bmod 5929) \equiv L(4467) \equiv 4466/77 \equiv 58 \bmod 77$$

Finally, multiplying the above result by μ completes the decryption process:

$$L(3222^{30} \bmod 5929)\cdot\mu \equiv 58 \cdot 74 \equiv 4292 \equiv 57 \equiv 42 + 15 \equiv m_1 + m_2 \bmod 77$$

Property 2: A full encryption of the second message is, in fact, unnecessary in the above property, and the r^n calculation in the second encryption can be left out while attaining the same results.

Formally: $D[E(m_1)\cdot g^{m_2} \bmod n^2] \equiv m_1 + m_2 \bmod n$

Proof: Let $c_1 \equiv g^{m_1}\cdot r_1^n \bmod n^2$; consider g^{m_2}; their product is $c_1 \cdot g^{m_2} \equiv g^{m_1}\cdot r_1^n \cdot g^{m_2} \bmod n^2$

$$\Rightarrow c_1 \cdot g^{m_2} \equiv g^{m_1+m_2}\cdot r_1^n \bmod n^2$$

Notice: the right hand side is, as far as decryption is concerned, exactly the same as the above right hand side. There is g raised to the (m_1+m_2) power, and an element of $\mathbb{Z}^*_n$ raised to the n – if this element is r_1, r_2, $r_1 \cdot r_2$, or some other r_i, it

doesn't matter, since in the first step of decryption, this element is raised to the $\lambda(n)\cdot n$ power, and goes to one by Carmichael's Theorem:

$$(c_1 \cdot g^{m_2})^{\lambda(n)} \equiv g^{\lambda(n)\cdot(m_1+m_2)} \cdot r_1^{n\cdot\lambda(n)} \equiv g^{\lambda(n)\cdot(m_1+m_2)} \bmod n^2$$

At this point it should be clear that the right hand side is exactly identical to the right hand side of raising $(c_1 \cdot c_2)$ to the $\lambda(n)$ in the above property, and so the rest of the decryption process is identical, and thus omitted. The result of continuing decryption from this point will be the same as above, and will result in attaining $(m_1+m_2) \bmod n$, thus making the decryption of the product of an encrypted message with a message essentially encrypted with $r = 1$, still $m_1 + m_2$.

Example: Assume the same public key as before: $(n, g) = (77, 5652)$; and the same two messages to send as before: $(m_1, r_1) = (42, 23)$ and $m_2 = 15$.

First, compute $E(m_1)$ and g^{m_2}, then take their product:

$$E(m_1) = c_1 = 4624$$

$$g^{m_2} \equiv 5652^{15} \equiv 5655 \bmod n^2$$

$$\Rightarrow E(m_1)\cdot g^{m_2} \equiv 4624 \cdot 5655 \equiv 26148720 \equiv 1830 \bmod 5929$$

Next, begin the decryption process by calculating $L([E(m_1)\cdot g^{m_2}]^{\lambda(n)} \bmod n^2)$:

$$L(1830^{30} \bmod 5929) \equiv L(4467) \equiv 4466/77 \equiv 58 \bmod 77$$

Finally, multiply this value by μ to complete the decryption process:

$$L(1830^{30} \bmod 5929)\cdot\mu \equiv 58 \cdot 74 \equiv 4292 \equiv 57 \equiv 42 + 15 \equiv m_1 + m_2 \bmod 77$$

<u>*Corollary to Property 2*</u>: Self-blinding – one can change the cipher text without changing the value of the original plaintext.

Formally: $D[E(m)\cdot g^{n\cdot x} \bmod n^2] \equiv n\cdot x + m \equiv m \bmod n$

Proof: This follows directly from Property 2 by letting $m_1 = m$, and $m_2 = n\cdot x$.

Discussion: Typically, since the final calculation in decrypting under the Paillier Cryptosystem is done mod n, messages must be broken into blocks such that each block, m, is an element of $\mathbb{Z}_n$. Since we don't actually care about the final reduction of $n\cdot x$, and, in fact, want it to go away mod n (as it does), we

don't need the above labeled m_2 to actually be a decryptable message, which is why it's okay for $n \cdot x \notin \mathbb{Z}_n$.

Notice, however, that the original cipher text, c, becomes a completely different cipher text, $c' \equiv c \cdot g^{n \cdot x} \bmod n^2$, but because $n \cdot x \equiv 0 \bmod n$, the original message, m, is still attained when c' is decrypted. An immediate consequence of this self-blinding property is error identification: if c and c' are sent over a noisy channel, where c' is a self-blinded copy of c, then a transmission error can be identified if c and c' do not decrypt to the same plaintext message. While this involves extra calculations to decrypt two cipher texts, it avoids the potential problem faced by just sending c twice, in which the same error may occur in both transmissions, leading to identical altered cipher texts being received twice, which would be interpreted as having received an error-free pair of cipher texts. If c and c' are both altered in transmission, they still should not decrypt to the same plaintext, indicating there is an error.

Example: Again, using the public key (n, g) = (77, 5652), send the same message (m, r) = (42, 23), and let x = 15 for blinding purposes.

E(m) has already been calculated in previous examples (4624), so begin by calculating $g^{n \cdot x}$, then taking its product with E(m):

$$g^{n \cdot x} \equiv 5652^{(77 \cdot 15)} \equiv 5652^{1155} \equiv 4115 \bmod 5929$$

$$E(m) \cdot g^{n \cdot x} \equiv 4624 \cdot 4115 \equiv 19027760 \equiv 1599 \bmod 5929$$

Next, begin the decryption process by calculating $L(E(m) \cdot g^{n \cdot x} \bmod n^2)$:

$$L(1599^{30} \bmod 5929) \equiv L(4852) \equiv 4851/77 \equiv 63 \bmod 77$$

Finally, multiplying by μ recovers the original plaintext message:

$$L(1599^{30} \bmod 5929) \cdot \mu \equiv 63 \cdot 74 \equiv 4662 \equiv 42 \equiv m \bmod 77$$

Notice: even though x was 15, which was the value of m_2 in Property 2, the final decryption is void of any sign of the 15, since multiplying it by n caused it to drop out mod n. Also note that, in Figure 4 of *§2.2 Decryption*, E(m) = 4624 was decrypted to yield m = 42. Here, $E(m) \cdot g^{n \cdot x} = 1599$ was also decrypted to attain m = 42. 4624 or 1599 could be sent as an encryption of m = 42,

showing that two completely different cipher texts can represent the same plaintext, when one cipher text is properly manipulated through self-blinding.

Property 3: Raising a cipher text to a constant power results in the constant multiple of the original plaintext.

Formally: $D[E(m)^k \bmod n^2] \equiv k \cdot m \bmod n$

Proof: Let $c \equiv g^m \cdot r^n \bmod n^2$, then $c = E(m)$, so $E(m)^k \equiv c^k \equiv (g^m \cdot r^n)^k \equiv g^{m \cdot k} \cdot r^{n \cdot k} \bmod n^2$

Beginning the decryption process, $(c^k)^{\lambda(n)} \equiv (g^{m \cdot k} \cdot r^{n \cdot k})^{\lambda(n)} \equiv g^{\lambda(n) \cdot m \cdot k} \cdot r^{\lambda(n) \cdot n \cdot k} \bmod n^2$

Notice: $r^{\lambda(n) \cdot n \cdot k} \equiv (r^k)^{\lambda(n) \cdot n} \bmod n^2$, where $r^k \in \mathbb{Z}^*_n$, so Carmichael's Theorem tells us that raising it to the $\lambda(n) \cdot n$ power makes it congruent to 1 mod n^2

So, $(c^k)^{\lambda(n)} \equiv g^{\lambda(n) \cdot m \cdot k} \equiv (1+n)^{\lambda(n) \cdot [\![g]\!]_{(1+n)} \cdot m \cdot k} \equiv 1 + m \cdot k \cdot \lambda(n) \cdot [\![g]\!]_{(1+n)} \cdot n \bmod n^2$

Then $L((c^k)^{\lambda(n)} \bmod n^2) \equiv m \cdot k \cdot \lambda(n) \cdot [\![g]\!]_{(1+n)} \bmod n$

And $\mu \cdot L((c^k)^{\lambda(n)} \bmod n^2) \equiv \lambda(n)^{-1} \cdot [\![g]\!]_{(1+n)}^{-1} \cdot m \cdot k \cdot \lambda(n) \cdot [\![g]\!]_{(1+n)} \equiv m \cdot k \bmod n$

Example: Still using the public key (n, g) = (77, 5652), and the same message (m, r) = (42, 23), let k = 93, so $k \notin \mathbb{Z}_n$ (the corollary will show the case when $k \in \mathbb{Z}_n$).

With E(m) still being 4624, begin by computing $E(m)^k$:

$$E(m)^k \equiv 4624^{93} \equiv 2990 \bmod 5929$$

Next, calculate $L(E(m)^k \bmod n^2)$ to begin decryption:

$$L(2990^{30} \bmod 5929) \equiv L(540) \equiv 539/77 \equiv 7 \bmod 77$$

Finally, multiply the above result by μ in order to complete decryption:

$$L(2990^{30} \bmod 5929) \cdot \mu \equiv 7 \cdot 74 \equiv 518 \equiv 56 \bmod 77 \equiv 3906 \equiv 93 \cdot 42 \bmod 77$$

Corollary to Property 3: Raising an encrypted message to the power of a second message results in the multiplication of plaintext messages.

Formally: $D[E(m_1)^{m_2} \bmod n^2] \equiv D[E(m_2)^{m1} \bmod n^2] \equiv m_1 \cdot m_2 \bmod n$

Proof: This property follows directly from Property 3 by letting $m = m_1$ and $k = m_2$ or by letting $m = m_2$ and $k = m_1$.

Discussion: This specific example of the above property, when $k \in \mathbb{Z}_n$, and thus can be considered a valid second message. Choosing a second message as the constant power results in a multiplication of plaintext messages.

Example: Once again, using the public key (n, g) = (77, 5652), encrypt the message $(m_1, r_1) = (42, 23)$, with the second message $m_2 = 15$.

Since $E(m_1)$ is still 4624, as previously calculated, begin by computing $E(m_1)^{m_2}$:

$$E(m_1)^{m_2} \equiv 4624^{15} \equiv 5391 \bmod 5929$$

Next, begin to decrypt $E(m_1)^{m_2}$ by applying the function L(u) to the previously attained value:

$$L(5391^{30} \bmod 5929) \equiv L(1618) \equiv 1617/77 \equiv 21 \bmod 77$$

Finally, multiplying by μ reveals the message contained in $E(m_1)^{m_2}$:

$$L(5391^{30} \bmod 5929)\cdot\mu \equiv 21\cdot 74 \equiv 1554 \equiv 14 \equiv 630 \equiv 15\cdot 42 \equiv m_1\cdot m_2 \bmod 77$$

4. Application To Electronic Voting

The first property listed above lends itself quite nicely to a potential application to a specific system of electronic voting. Consider a simple voting scheme in which an item up for debate can either be supported or opposed. Each voter casts his/her vote using the Paillier Cryptosystem, such that a vote in favour of the proposed item is the plaintext message 1, and a vote against the item is the message 0. Each voter chooses a random r to encrypt his/her vote with, but they all use the same officially designated public key (n, g). A tally, x, is kept of how many votes are cast, then all the encrypted votes are multiplied together, and the result is decrypted as some value, y. Since multiplying encrypted text results in the addition of plaintexts, this process will result in the addition of 1's and 0's as y, effectively tallying all of the votes for the proposition. Since the number of votes cast, x, is known, this leaves x–y votes against the proposition, and the vote can be properly decided. Notice that a vote of zero, or 'against', will not automatically encrypt to zero, as $g^0 r^n \equiv r^n \bmod n^2$, and r is a nonzero value, as it is a unit in $\mathbb{Z}^*_n$. Since g^1 is a constant, set value, as is g^0, the difference in cipher text values for people casting the same vote is determined by r^n, of which n is also a fixed value. So there does exist potential for two voters to encrypt their votes and have the same resulting cipher text value, however, the likelihood of casting the same vote and choosing the same random r is small, given a large enough choice for n.

Even if two votes are cast the same and encrypted with the same r value, and assuming each voter can see his/her own cipher text value (which is not necessary, but may be displayed to present a worst-case scenario, privacy-wise), the voters' privacy is still protected. Assuming voters don't get to see the encrypted values of their peers' votes (so one person can't say "oh, we have the same cipher text, we must have voted the same"), and assuming the method for collecting and multiplying the cipher texts is either automatic, or conducted by someone who does not cast a vote (a person from outside the voting district, for example), the slight potential of a person recognizing a cipher text as matching his/her own, thus divulging the voter's choice, is practically eliminated. Yet another step to ensure voter privacy could be to automatically apply self-blinding to each vote once it is cast. Then the voter may know what s/he encrypted his/her vote to, but s/he does not see what the resulting cipher text is after the vote is blinded using a rotating x value for $g^{n \cdot x}$.

This simple system works well when the voter is presented with only two choices, however it would not work for say a general public election in which third parties create more than two potential candidates.

5. SUMMARY

The Paillier Cryptosystem works, in large, due to the function ε_g: $\mathbb{Z}_n \times \mathbb{Z}^*_n \rightarrow \mathbb{Z}^*_{n^2}$, which maps $(x, y) \rightarrow g^x \cdot y^n \bmod n^2$. When the choice of x has an order in $\mathbb{Z}_n$ that is a nonzero multiple of n, ε_g is a bijection, which is what allows for decryption. Because messages are encrypted as exponents, and the exponents are then retrieved during decryption mod n, several interesting properties arise, such as self-blinding ($D[E(m)g^{n \cdot x} \bmod n^2] \equiv n \cdot x + m \equiv m \bmod n$) and the multiplication of cipher texts yielding an addition of plaintexts ($D[E(m_1) \cdot E(m_2) \bmod n^2] \equiv m_1 + m_2 \bmod n$). These properties have potential for application to real world systems, such as the previously described electronic voting scheme, which works entirely on the basis of being able to manipulate cipher texts in a controlled fashion so as to have a known affect on the original plaintext messages.

Overall, the Paillier Cryptosystem is rather efficient: encryption is, more or less, two exponentiations mod n^2, and decryption, as mentioned at the end of *§2.3 Mathematics in the Cryptosystem*, is, essentially, one exponentiation mod n^2. The properties the cryptosystem possesses are also cause for intrigue, as the Paillier Cryptosystem can be used in ways that many other cryptosystem simply can't be used. The electronic voting system described is

merely one imagined application of the system, and certainly its properties could be useful in other applications as well. Thus, the Paillier Cryptosystem is worthwhile to consider both for the mathematics behind it, as well as for its potential real world applications.

6. BIBLIOGRAPHY

[1] De Vries, Andreas. "Carmichael Function." Math IT. 2002. 14 Apr. 2008 <http://www.math-it.org/Mathematik/Zahlentheorie/Carmichael.html>.

[2] Koshy, Thomas. "Properties of Function." Discrete Mathematics with Applications. N.p.: Academic Press, 2004. 139.

[3] Paillier, Pascal. "Public-Key Cryptosystems Based on Composite Degree Residuosity Classes." Advances in Cryptology – EUROCRYPT '99 1592 (1999): 223-238. 15 Apr. 2008 <http://www.gemplus.com/smart/rd/publications/pdf/Pai99pai.pdf>.

Hash Functions

- condenses arbitrary message to fixed size

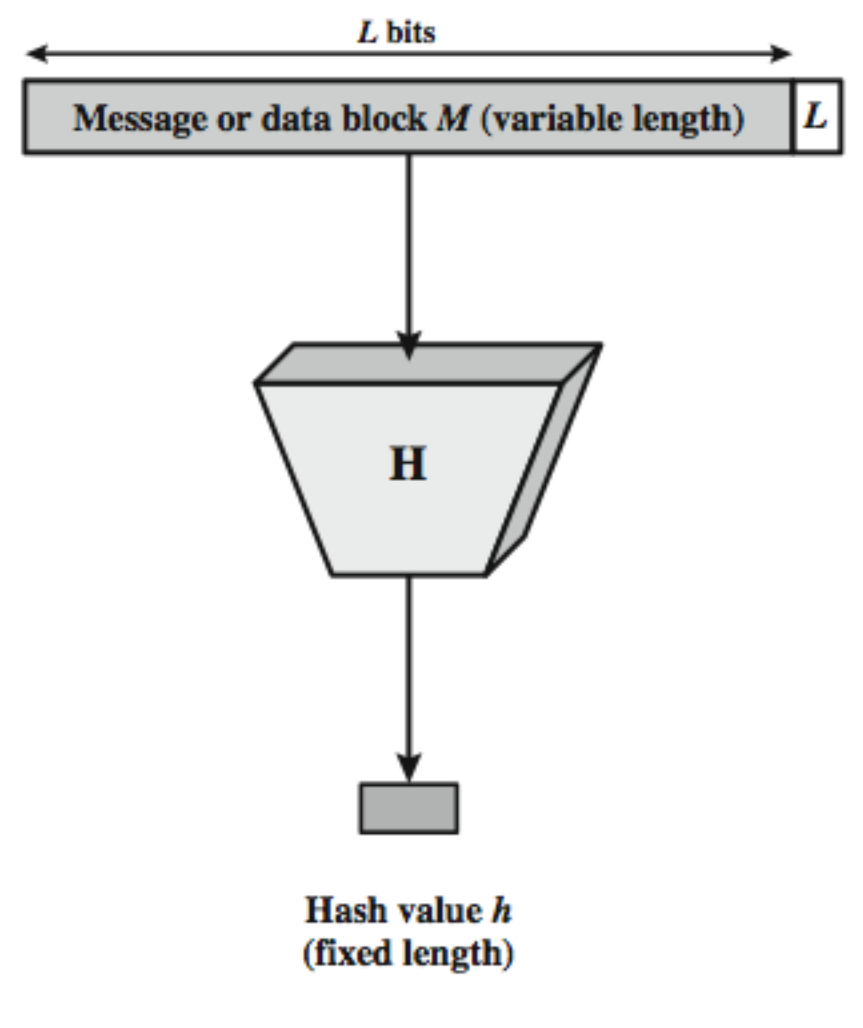
  `h = H(M)`

- usually assume hash function is public
- hash used to detect changes to message
- want a cryptographic hash function
  - computationally infeasible to find data mapping to specific hash (one-way property)
  - computationally infeasible to find two data to same hash (collision-free property)

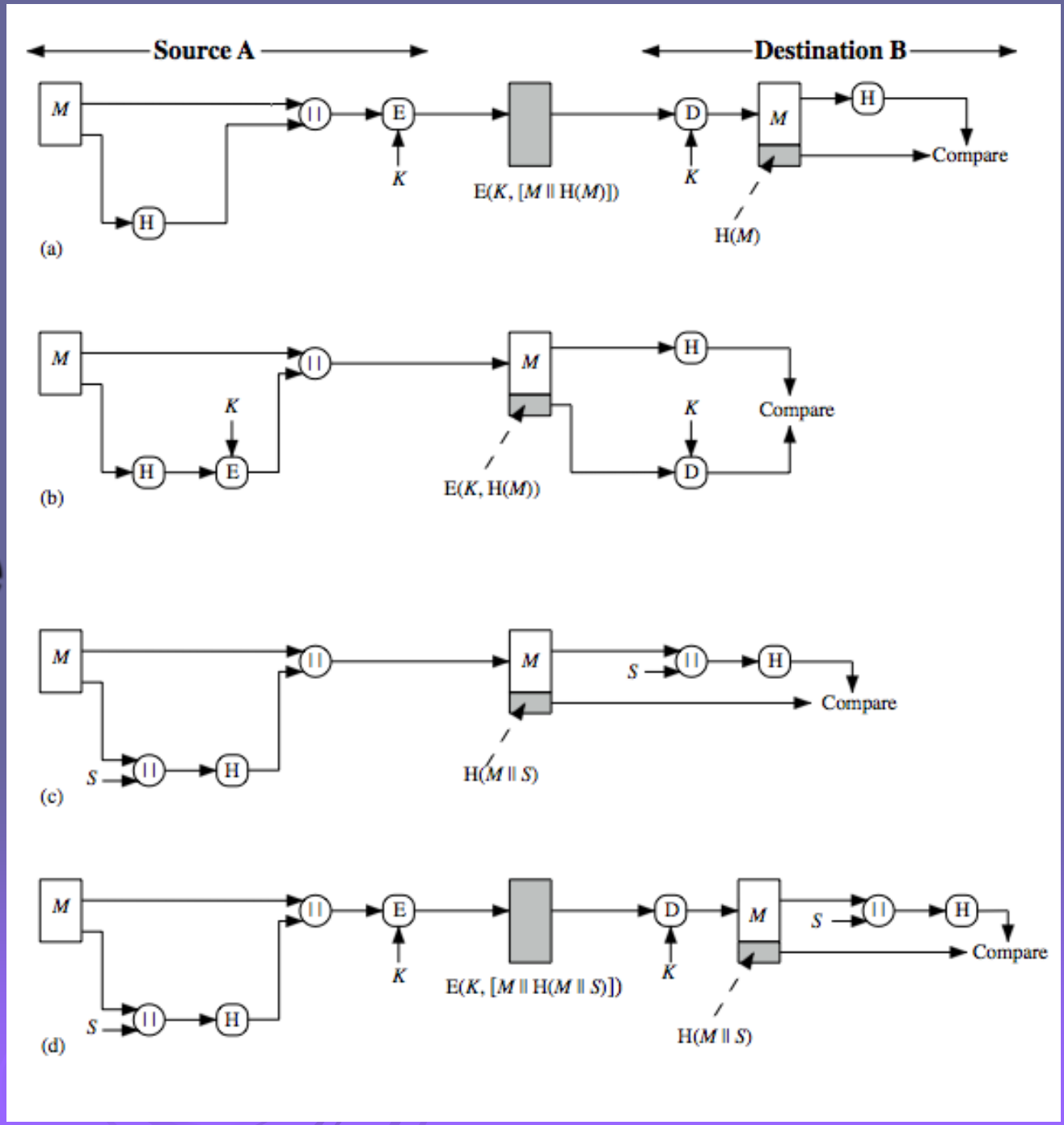
Cryptographic Hash Function

- Crypto hash function h(x) must provide
  - **Compression** — output length is small
  - **Efficiency** — h(x) easy to compute for any x
  - **One-way** — given a value y it is infeasible to find an x such that h(x) = y
  - **Weak collision resistance** — given x and h(x), infeasible to find $y \neq x$ such that h(y) = h(x)
  - **Strong collision resistance** — infeasible to find any x and y, with $x \neq y$ such that h(x) = h(y)
- Many collisions exist, but cannot find any

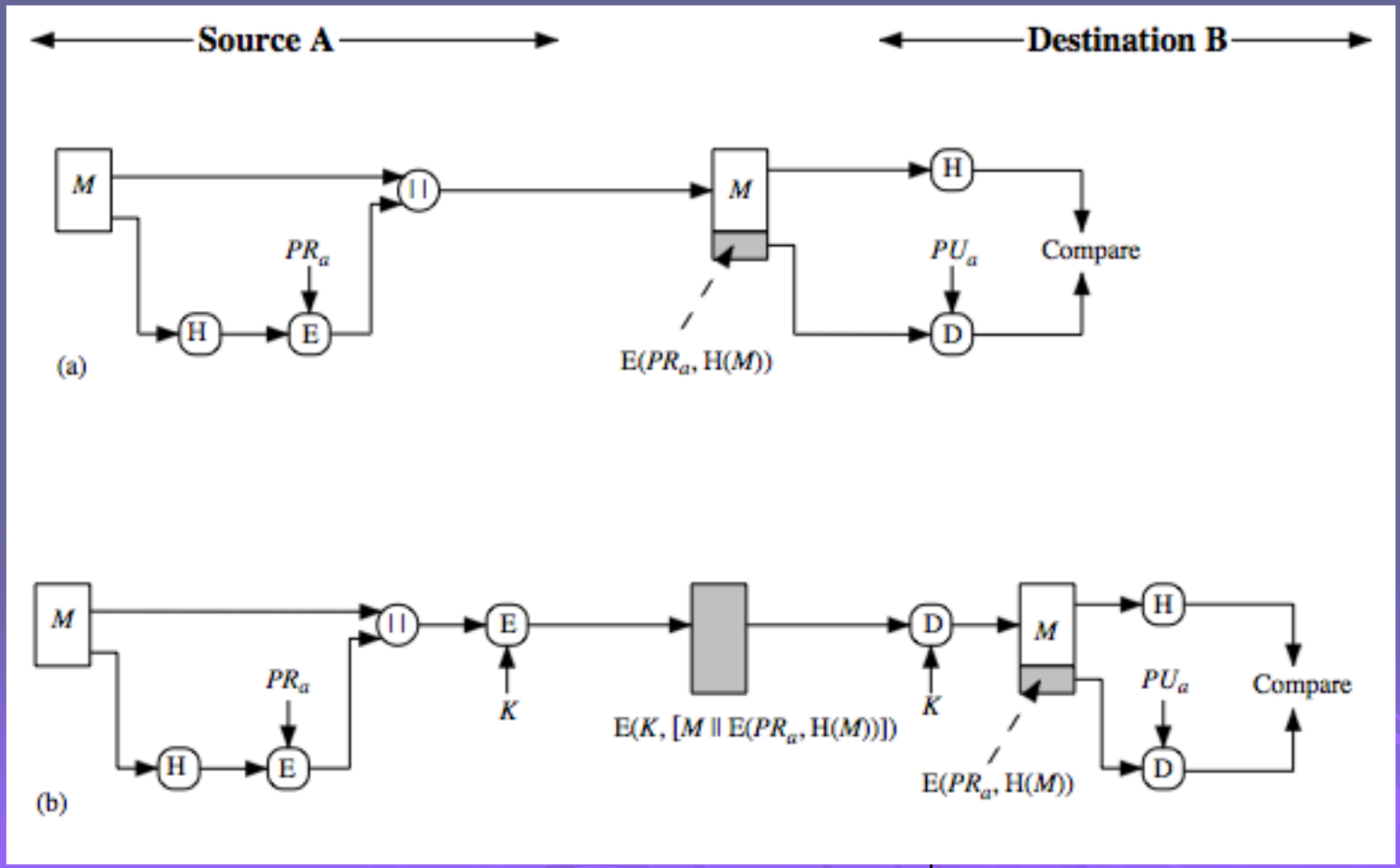
Cryptographic Hash Function



Hash Functions & Message Authent-ication



Hash Functions & Digital Signatures



Other Hash Function Uses

- to create a one-way password file
  - store hash of password not actual password
- for intrusion detection and virus detection
  - keep & check hash of files on system
- pseudorandom function (PRF) or pseudorandom number generator (PRNG)

Hash Function Requirements

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	H(x) is relatively easy to compute for any given x, making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h, it is computationally infeasible to find y such that H(y) = h.
Second preimage resistant (weak collision resistant)	For any given block x, it is computationally infeasible to find y ! x with H(y) = H(x).
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) such that H(x) = H(y).
Pseudorandomness	Output of H meets standard tests for pseudorandomness

Attacks on Hash Functions

- have brute-force attacks and cryptanalysis
- a preimage or second preimage attack
  - find y s.t. $H(y)$ equals a given hash value
- collision resistance
  - find two messages x & y with same hash so $H(x) = H(y)$
- hence value $2^{m/2}$ determines strength of hash code against brute-force attacks
  - 128-bits inadequate, 160-bits suspect

IV054 Chapter 9: Digital signatures

Digital signatures are one of the most important inventions of modern cryptography.

The problem is how can a user sign a message such that everybody (or the intended addressee only) can verify the digital signature and the signature is good enough also for legal purposes.

Assume that all users use a public-key cryptosystem.

Signing a message w by a user A so that any user can verify the signature;

$$d_A(w)$$

Signing a message w by a user A so that only user B can verify the signature;

$$e_B(d_A(w))$$

Sending a message w and a signed message digest of w obtained by using a hash function standard h:

$$(w, d_A(h(w)))$$

Example Alice succeeds after 20 years to factor the integer, Bob used, as modulus, to sign documents, using RSA, 20 years ago. Even if the key is already expired, she can write Bob's will, leaving fortune to Alice, and date it 20 years ago.

Moral: It may pay of to factor a single integers using many years of many computer power.

Digital signatures – basic goals

Digital sigantures should be such that each user should be able to verify signatures of other users, but that should give him/her no information how to sign a message on behind of other users.

The main difference from a handwritten signature is that digital signature of a message is intimately connected with the message, and for different messages is different, whereas the handwritten signature is adjoined to the message and always looks the same.

Technically, digital signature is performed by a signing algorithm and it is verified by a verification algorithm.

A copy of digital (conventional) signature is identical (usually distinguishable) to (from) the origin. A care has therefore to be made that a classical signature is not misused.

This chapter contains an overview of the main techniques for design and verification of digital signatures (as well as some attacks to them).

Digital signatures

If only signature (but not the encryption of the message) are of importance, then it suffices that Alice sends to Bob

$$(w, d_A(w))$$

Caution: Signing a message w by A for B by

$$e_B(d_A(w))$$

is O.K., bat the symmetric solution with encoding was

$$c = d_A(e_B(w))$$

is not good.

An active enemy, the tamperer, can intercept the message, then compute

$$d_T(e_B(c)) = d_T(e_A(w))$$

and send it to Alice, pretending it is from him (without being able to decrypt the message).

Any public-key cryptosystem in which the plaintext and cryptotext spaces are the same can be used for digital signature.

IV054 DIGITAL SIGNATURE of ONE BIT

Let us start with a very simple but much illustrating (though non-practical) example how to sign a single bit.

Design of the digital signature scheme

A one-way function f(x) is chosen.

Two integers k_0 and k_1 are chosen, kept secret and

$$f, (0, s_0), (1, s_1)$$

are made public, where

$$s_0 = f(k_0),\ s_1 = f(k_1)$$

Signature of a bit b

$$(b, k_b).$$

Verification of the signature

$$s_b = f(k_b)$$

SECURITY?

RSA signatures and their attacks

Let us have an RSA cryptosystem with encryption and decryption exponents *e* and *d*.

Signing of a message *w* $s = (w, \sigma)$, where $\sigma = w^d$

Verification of a signature $s = (w, \sigma)$:

$$w = \sigma^e$$

Attacks

• It might happen that Bob accepts a signature not produced by Alice. Indeed, let Eve, using Alice's public key, computes w^e and says that (w^e, *w*) is a message signed by Alice.

Everybody verifying Alice's signature gets $w^e = w^e$.

• Some new signatures can be produced without knowing secret key.

Indeed, is σ_1 and σ_2 are signatures for w_1 and w_2, then $\sigma_1\sigma_2$ and σ_1^{-1} are signatures for $w_1 w_2$ and w_1^{-1}.

ENCRYPTION versus SIGNATURE

Cryptosystem: Let each user U uses a cryptosystem with encryption and decryption algorithms: e_U, d_U

Message: w

PUBLIC-KEY CRYPTOGRAPHY

Encryption: $e_U(w)$

Decryption: $d_U(e_U(w))$

PUBLIC-KEY SIGNATURES

Signing: $d_U(w)$

Verification of signatures: $e_U(d_U(w))$

DIGITAL SIGNATURE SYSTEMS

A digital signature system (DSS) consists:

- P - the space of possible plaintexts (messages).
- S - the space of possible signatures.
- K - the space of possible keys.

- For each $k \in K$ there is a signing algorithm $sig_k \in S_a$ and a corresponding verification algorithm $ver_k \in V$ such that

- $sig_k : P \rightarrow S$.

- $ver_k : P \otimes S \rightarrow \{true, false\}$

and

$$ver_k(w,s) = \begin{cases} \text{true, if } s = sig(w); \\ \text{false, otherwise.} \end{cases}$$

Algorithms sig_k and ver_k should be computable in polynomial time.

Verification algorithm can be publically known; signing algorithm (actually only its key) should be kept secret.

FROM PKC to DSS

Any public-key cryptosystem in which the plaintext and cryptotext space are the same can be used for digital signature.

Signing of a message w by a user A so that any user can verify the signature:

$$d_A(w).$$

Signing of a message w by a user A so that only user B can verify the signature;

$$e_B(d_A(w)).$$

Sending of a message w and a signed message digest of w obtained by using a (standard) hash function h:

$$(w, d_A(\mathrm{h}(w))).$$

If only signature (but not the encryption of the message) are of importance, then it suffices that Alice sends to Bob

$$(w, d_A(w)).$$

FROM PKC to DSS - additions

Caution! Signing a message w by A for B by

$$e_B\,(d_A\,(w))$$

is O.K., but a symmetric solution with encoding as

$$c = d_A\,(e_B\,(w))$$

is not good. Indeed, an active enemy, the tamperer, can intercept the message, then compute

$$d_T\,(e_A\,(c)) = d_T\,(e_B\,(w))$$

and send it to Bob, pretending it is from him (without being able to decrypt the message).

Example: Alice succeeds after 20 years to factor the integer that Bob used, as modulus, to sign his will, using RSA, 20 years ago. Even if the key has already expired, she can rewrite Bob's will, leaving fortune to Alice, and date it 20 years ago.

Moral: It may pay of to factor a single integers using many years of many computer power.

ElGamal signatures

Design of ElGamal digital siganture system: choose: prime p, integers $1 < q < x < p$, q be a primitive element of Z_p^*;

Compute: $y = q^x \bmod p$

key $K = (p, q, x, y)$
public key (p, q, y) - **trapdoor**: x

Signature of a message w: Let $r \in Z_{p-1}^*$ be randomly chosen and kept secret.

$$sig(w, r) = (a, b),$$

where $a = q^r \bmod p$

and $b = (w - xa)r^{-1} \pmod{p-1}$.

Verification: accept a signature (a,b) of w as valid if

$$y^a a^b \equiv q^w \pmod{p}$$

(Indeed: $y^a a^b \equiv q^{ax} q^{rb} \equiv q^{ax + w - ax + k(p-1)} \equiv q^w \pmod{p}$)

ElGamal signatures

Example choose: $p = 11$, $q = 2$, $x = 8$

compute: $y = 2^8 \bmod 11 = 3$

Signing of $w = 5$,

choose $k = 9$ - O.K. because gcd(9, 10) = 1

compute $a = 2^9 \bmod 11 = 6$

solve equation: $5 \equiv 8 \cdot 6 + 9b \pmod{10}$

that is $7 \equiv 9b \pmod{10} \Rightarrow b=3$

signature: (6, 3)

Security of ElGamal signatures

Let us analyze several ways an eavesdropper Eve can try to forge ElGamal signature (with x - secret; p, q and $y = q^x \bmod p$ - public):

$$sig(w, r) = (a, b);$$

where r is random and $a = q^r \bmod p$; $b = (w - xa)r^{-1} \pmod{p-1}$.

1. First suppose Eve tries to forge signature for a new message w , without knowing x.

• If Eve first chooses a value a and tries to find the corresponding b, it has to compute the discrete logarithm

$$\lg_a q^w y^{-a},$$

because $a^b \equiv q^{r(w - xa)r^{(-1)}} \equiv q^{w - xa} \equiv q^w y^{-a}$.

• If Eve first chooses b and then tries to find a, she has to solve the equation

$$y^a a^b \equiv q^{xa} q^{rb} \equiv q^w \pmod{p}.$$

It is not known whether this equation can be solved for a efficiently.

2. If Eve chooses a and b and tries to determine w, then she has to compute discrete logarithm

$$\lg_q y^a a^b.$$

Hence, Eve can not sign a “random” message this way.

Forging and misusing of ElGamal signatures

There are ways how to produce, using ElGamal signature scheme, validforged signatures, but they do not allow an opponent to forge signatures on messages of his/her choice.

For example, if $0 \le i, j \le p-2$ and $\gcd(j, p-1) = 1$, then for

$$a = q^i y^j \bmod p;\ b = -aj^{-1} \bmod (p-1);\ \ w = -aij^{-1} \bmod (p-1)$$

the pair

(a, b) is a valid signature of the message w.

This can be easily shown by checking the verification condition.

There are several ways ElGamal signatures can be broken if they are used not carefully enough.

For example, the random r used in the signature should be kept secret. Otherwise the system can be broken and signatures forged. Indeed, if r is known, then x can be computed by

$$x = (w - rb)\, a^{-1} \bmod (p-1)$$

and once x is known Eve can forge signatures at will.

Another misuse of the ElGamal signature system is to use the same r to sign two messages. In such a case x can be computed and system can be broken.

Digital Signature Standard

In December 1994, on the proposal of the National Institute of Standards and Technology, the following Digital Signature Algorithm (DSA) was accepted as a standard.

Design of DSA

1. The following global public key components are chosen:

- p - a random l-bit prime, 512 Ł l Ł 1024, $l = 64k$.
- q - a random 160-bit prime dividing p -1.
- $r = h^{(p-1)/q} \bmod p$, where h is a random primitive element of Z_p, such that r>1 $r \neq 1$

(observe that r is a q-th root of 1 mod p).

2. The following user's private key components are chosen:

- x - a random integer, $0 < x < q$,
- $y = r^x \bmod p$.

3. Key is $K = (p, q, r, x, y)$

Signing and Verification

Signing of a 160-bit plaintext w

- choose random $0 < k < q$ such that $\gcd(k, q) = 1$
- compute $a = (r^k \bmod p) \bmod q$
- compute $b = k^{-1}(w + xa) \bmod q$ where $kk^{-1} \equiv 1 \pmod q$
- signature: $sig(w, k) = (a, b)$

Verification of signature (a, b)

- compute $z = b^{-1} \bmod q$
- compute $u_1 = wz \bmod q$,

  $u_2 = az \bmod q$

verification:

$$ver_K(w, a, b) = \text{true} \iff (r^{u1} y^{u2} \bmod p) \bmod q = a$$

From ElGamal to DSA

DSA is a modification of ElGamal digital signature scheme. It was proposed in August 1991 and adopted in December 1994.

Any proposal for digital signature standard has to go through a very careful scrutiny. Why?

Encryption of a message is usually done only once and therefore it usually suffices to use a cryptosystem that is secure **at the time of the encryption**.

On the other hand, a signed message could be a contract or a will and it can happen that it will be needed to verify a signature **many years after the message is signed**.

Since ElGamal signature is no more secure than discrete logarithm, it is necessary to use large p, with at least 512 bits.

However, with ElGamal this would lead to signatures with at least 1024bits what is too much for such applications as smart cards.

In DSA a 160 bit message is signed using 320-bit signature, but computation is done modulo with 512-1024 bits.

Observe that y and a are also q-roots of 1. Hence any exponents of r,y and a can be reduced module q without affecting the verification condition.

This allowed to change ElGamal verification condition: $y^{a} a^{b} = q^{w}$.

Fiat-Shamir signature scheme

Choose primes p, q, compute $n = pq$ and choose:
as public key $v_1,\ldots,v_k$ and compute secret key $s_1,\ldots,s_k, s_i = \sqrt{v_i^{-1}} \bmod n.$

Protocol for Alice to sign a message w:

(1) Alice chooses t random integers $1 \le r_1,\ldots,r_t < n$, computes $x_i = r_i^2 \bmod n$, $1 \le i \le t$.

(2) Alice uses a publically known hash function h to compute

$$H = h(w x_1 x_2 \ldots x_t)$$

and then uses first kt bits of H, denoted as b_{ij}, $1 \le i \le t$, $1 \le j \le k$ as follows.

(3) Alice computes $y_1,\ldots,y_t$ $\quad y_i = r_i \prod_{j=1}^{k} s_j^{b_{ij}} \bmod n$

(4) Alice sends to Bob w, all b_{ij} and all y_i
{ Bob already knows Alice's public key $v_1,\ldots,v_k$ } and also h

(5) Bob computes $z_1,\ldots,z_k$ $\quad z_i = y_i^2 \prod_{j=1}^{k} v_j^{b_{ij}} \bmod n = r_i^2 \prod_{j=1}^{k} \left(v_j^{-1}\right)^{b_{ij}} \prod_{j=1}^{k} v_j^{b_{ij}} = r_i^2 = x_i$

and verifies that the first $k \times t$ bits of $h(w x_1 x_2 \ldots x_t)$ are the b_{ij} values that Alice sent to him.

Security of this signature scheme is 2^{-kt}.

Advantage over the RSA-based signature scheme: only about 5% of modular multiplications is needed.

SAD STORY

Alice and Bob got to jail – and unfortunately to different jails.

Walter, the warden, allows them to communicate by network, but he will not allow that their messages are encrypted.

Problem: Can Alice and Bob set up a subliminal channel, a covert communications channel between them, in full view of Walter, even though the messages themselves that they exchange contain no secret information?

IV054 Ong-Schnorr-Shamir subliminal channel scheme

Story Alice and Bob are in different jails. Walter, the warden, allows them to communicate by network, but he will not allow messages to be encrypted. Can they set up a subliminal channel, a covert communications channel between them, in full view of Walter, even though the messages themselves contain no secret information?

Yes. Alice and Bob create first the following communication scheme:

They choose a large n and an integer k such that $\gcd(n, k) = 1$.

They calculate $h = k^{-2} \bmod n = (k^{-1})^2 \bmod n$.

Public key: h, n

Trapdoor information: k

Let secret message Alice wants to send be w (it has to be such that $\gcd(w, n) = 1$)

Denote a harmless message she uses by w' (it has to be such that $\gcd(w', n) = 1$)

Signing by Alice:

$$S_1 = \frac{1}{2} \cdot \left(\frac{w'}{w} + w\right) \bmod n$$

$$S_2 = \frac{1}{2} \cdot \left(\frac{w'}{w} - w\right) \bmod n$$

Signature: (S_1, S_2). Alice then sends to Bob (w', S_1, S_2)

Signature verification by Walter: $w' = S_1^2 - hS_2^2 \pmod n$

Decryption by Bob:

$$w = \frac{w'}{(S_1 + k^{-1} S_2)} \bmod n.$$

One-time signatures

Lamport signature scheme shows how to construct a signature scheme for one use from any one-way function.

Let k be a positive integer and let $P = \{0,1\}^k$ be the set of messages.
Let $f: Y \rightarrow Z$ be a one-way function and let Y be the set of` "signatures".

For $1 \le i \le k$, $j = 0,1$ let $y_{ij} \in Y$ be chosen randomly and $z_{ij} = f(y_{ij})$.

The key K consists of $2k$ y's and z's. y's are secret, z's are public.

Signing of a message $x = x_1 \ldots x_k \in \{0,1\}^k$

$$sig(x_1 \ldots x_k) = (y_{1,x1}, \ldots, y_{k,xk}) = (a_1, \ldots, a_k)$$ - notation

and

$$ver_K(x_1 \ldots x_k, a_1, \ldots, a_k) = \text{true} \iff f(a_i) = z_{i,xi},\ 1 \le i \le k$$

Eve cannot forge a signature because she is unable to invert one-way functions.

Important note: Lampert signature scheme can be used to sign only one message.

UNDENIABLE SIGNATURES I

Undeniable signatures are signatures that have two properties:

- A signature can be verified only at the cooperation with the signer – by means of a challenge-and-response protocol.
- Signer cannot deny a correct signature. To achieve that steps are a part of the protocol that force the signer to cooperate – by means of a disavowal protocol – this protocol makes possible to prove the invalidity of a signature and to show that it is a forgery. (If the signer refuses to take part in the disavowal protocol, then the signature is considered to be genuine.)

Undeniable signature protocol of Chaum and van Antwerpen (1989), discussed next, is again based on infeasibility of the computation of the discrete logarithm.

Undeniable signatures II

Undeniable signatures consist:

- Signing algorithm
- Verification protocol, that is a *challenge-and-response protocol*.

In this case it is required that a signature cannot be verified without a cooperation of the signer (Bob).

This protects Bob against the possibility that documents signed by him are duplicated and distributed without his approval.

- Disavowal protocol, by which Bob can prove that a signature is aforgery.

This is to prevent Bob from disavowing a signature he made at an earlier time.

Chaum-van Antwerpen undeniable signature schemes (CAUSS)

- p, r are primes $p = 2r + 1$
- $q \in Z_p^*$ is of order r;
- $1 \leq x \leq r - 1$, $y = q^x \bmod p$;
- G is a multiplicative subgroup of Z_p^* of order q (G consists of quadratic residues modulo p).

Key space: $K = \{p, q, x, y\}$; p, q, y are public, $x \in G$ is secret.

Signature: $s = sig_k(w) = w^x \bmod p$.

Fooling and Disallowed protocol

Since it holds:

Theorem If $s \neq w^x \bmod p$, then Alice will accept s as a valid signature for w with probability $1/r$.

Bob cannot fool Alice except with very small probability and security is unconditional (that is, it does not depend on any computational assumption).

Disallowed protocol

Basic idea: After receiving a signature s Alice initiates two independent and unsuccessful runs of the verification protocol. Finally, she performs a “consistency check" to determine whether Bob has formed his responses according to the protocol.

- Alice $e_1, e_2 \in Z_r^*$.
- Alice computes $c = s^{e1}y^{e2} \bmod p$ and sends it to Bob.
- Bob computes $d = c^{x\wedge(-1) \bmod r} \bmod p$ and sends it to Alice.
- Alice verifies that $d \neq w^{e1}q^{e2} \pmod p$.
- Alice $f_1, f_2 \in Z_r^*$.
- Alice computes $C = s^{f1}y^{f2} \bmod p$ and sends it to Bob.
- Bob computes $D = C^{x\wedge(-1) \bmod r} \bmod p$ and sends it to Alice.

Fooling and Disallowed protocol

- Alice verifies that $D \neq w^{f1} q^{f2} \pmod{p}$.
- Alice concludes that s is a forgery iff

$$(dq^{-e2})^{f1} \equiv (Dq^{-f2})^{e1} \pmod{p}.$$

CONCLUSIONS

It can be shown:

Bob can convince Alice that an invalid signature is a forgery. In order to that it is sufficient to show that if $s \neq w^{x}$, then

$$(dq^{-e2})^{f1} \equiv (Dq^{-f2})^{e1} \pmod{p}$$

what can be done using congruency relation from the design of thesignature system and from the disallowed protocol.

Bob cannot make Alice believe that a valid signature is aforgery, except with a very small probability.

Signing of fingerprints

Signatures scheme presented so far allow to sign only "short" messages. For example, DSS is used to sign 160 bit messages (with 320-bit signatures).

A naive solution is to break long message into a sequence of shortones and to sign each block separately.

Disadvantages: signing is slow and for long signatures integrity is not protected.

The solution is to use fast public ***hash functions h*** which maps a message of any length to a fixed length fingerprint. The fingerprint is then signed.

Scheme:

message	w	arbitrary length
message digest	$z = h(w)$	160bits
signature	$y = sig(z)$	320bits

If Bob wants to sign a message w he sends $(w, sig(h(w)))$.

Collision-free hash functions revisited

For a hash function it is necessary to be good enough for creating fingerprints that do not allow various forgeries of signatures.

For example, Eve can start with a valid signature (w, $sign(h(w))$), compute $h(w)$ and try to find w' such that $h(w) = h(w')$. If she succeeds, then

$$(w', sign(h(w)))$$

is a valid signed message, a **forgery**.

In order to prevent the above type of attacks, and some other, it is required that a hash function h satisfies the following **collision-free property**.

Definition A hash function h is (strongly) collision-free if it is computationally infeasible to find messages w and w' such that $h(w) = h(w')$.

Another possible attack: Eve computes a signature y on a random fingerprint z and then find an x such that $z = h(x)$. In such a case (x,y) is a valid signature.

In order to prevent the above attack, it is required that in signatures we use one-way hash functions.

It is not difficult to show that for hash-functions (strong) collision property implies the one-way property.

Collision-free hash functions revisited

Theorem Let $h:X \to Z$ be a hash function where $|X|$ and $|Z|$ are finite and $|X| \geq 2|Z|$. Suppose that A is an inversion algorithm for h. Then there exists a Las Vegas algorithm to find a collision for h with probability at least 1/2.

Timestamping

There are various ways that a signing algorithm can be compromised.

For example: if Eve determines the secret key of Bob, then she can forge signatures on any Bob's message she likes. If this happens, authenticity of all messages signed by Bob before Eve got the secret key is to be questioned.

The key problem is that there is no way to determine when a message was signed.

A timestamping should provide proof that a message was signed at a certain time.

A method for timestamping of signatures:

In the following *pub* denotes some publically known information that could not be predicted before the day of the signature (for example, stock-market data).

Timestamping by Bob of a signature on a message *w*.

- Bob computes $z = h(w)$;
- Bob computes $z' = h(z \,||\, pub)$.
- Bob computes $y = sig(z')$.
- Bob publishes (z, pub, y) in the next days's newspaper.

It is now clear that signature could not be done after triple (x, pub, y) was published, but also not before date *pub* was known.

BLIND SIGNATURES

The basic idea is that a Sender makes a Signer to sign a message *m* without knowing *m*, therefore blindly – this is needed in e-commerce.

Blind signing can be achieved by a two party protocol, between the Sender and the Signer, that has the following properties.

- In order to sign (by a Signer) a message *m*, the Sender computes, using a blinding procedure, from *m* an *m** from which *m* can not be obtained without knowing a secret, and sends *m** to the Signer.
- The Signer signs *m** to get a signature s_{m^*} (of m*) and sends s_{m^*} to the Sender. Signing is done in such a way that the Sender can afterwards compute, using an unblinding procedure, from Signer's signature s_{m^*} of *m** -- the signer signature s_m of *m*.

CHUM's BLIND SIGNATURE

This blind signature protocol combines RSA with blinding/unblinding features.

Bob's RSA public key is (n,e) and his private key is d.

Let m be a message, $0 < m < n$,

PROTOCOL:

- Alice chooses a random $0 < k < n$ with $gcd(n,k)=1$.
- Alice computes $m^* = mk^e \ (\textbf{mod}\ n)$ and sends it to Bob (this way Alice blinds the message m).
- Bob computed $s^* = (m^*)^d (\textbf{mod}\ n)$ and sends s^* to Alice (this way Bob signs the blinded message m^*).
- Alice computes $s = k^{-1}s^* (\textbf{mod}\ n)$ to obtain Bob's signature m^d of m (Alice performs unblinding of m*).

Verification is equivalent to that of RSA signature scheme.

FAIL-THEN-STOP SIGNATURES

They are signatures schemes that use a trusted authority and provide ways to prove, if it is the case, that a powerful enough adversary is around who could break the signature scheme and therefore its use should be stopped.

The scheme is maintained by a trusted authority that chooses a secret key for each signer, keep it secret, even from the signers themselves, and announces only the related public key.

An important idea is that signing and verification algorithms are enhanced by a so called proof of forgery algorithm. When the signer see a forged signature he is able to compute his secret key and by submitting it to the trusted authority to prove the existence of a forgery and to achieve that any further use of the signature scheme is used.

So called Heyst-Pedersen Scheme is an example of a Fail-Then-Stop siganture Scheme.

IV054 Digital signature with encryption and resending

1. Alice signs the message: $s_A(w)$.
2. Alice encrypts the signed message: $e_B(s_A(w))$.
3. Bob decrypt the signed message: $d_B(e_B(s_A(w))) = s_A(w)$.
4. Bob verifies signature and recovers the message $v_A(s_A(w)) = w$.

Resending the message as a receipt

5. Bob signs and encrypts the message and sends $e_A(s_B(w))$.
6. Alice decrypts the message and verifies the signature.

Assume now: $v_x = e_x$, $s_x = d_x$ for all users x.

A surprising attack to the previous scheme

1. Mallot intercept $e_B(s_A(w))$.
2. Later Mallot sends $e_B(s_A(w))$ to Bob pretending it is from him (from Mallot).
3. Bob decrypts and "verifies" the message by computing

   $$e_M(d_B(c_B(d_A(w)))) = e_M(d_A(w))$$ - a garbage.
4. Bob goes on with the protocol and sends Mallot the receipt:

   $$e_M(d_B(e_M(d_A(w))))$$
5. Mallot gets w. Indeed, $e_A(d_M(e_B(d_M(e_M(d_B(e_M(d_A(w)))))))) = w$.

IV054 A MAN-IN-THE-MIDDLE-ATTACK

Consider the following protocol:

1. Alice sends Bob the pair $(e_B(e_B(w)A), B)$ to B.
2. Bob uses d_B to get A and w, and acknowledges by sending the pair $(e_A(e_A(w)B), A)$ to Alice.

(Here the function e and d are assumed to operate on numbers, names $A,B,\ldots$ are sequences of digits and $e_B(w)A$ is a sequence of digitals obtained by concatenating $e_B(w)$ and A.)

What can an active eavesdropper *C* do?

- C can learn $(e_A(e_A(w)\,B), A)$ and therefore $e_A(w')$, $w' = e_A(w)B$.
- C can now send to Alice the pair $(e_A(e_A(w')\,C), A)$.
- Alice, thinking that this is the step 1 of the protocol, acknowledges by sending the pair $(e_C(e_C(w')\,A), C)$ to C.
- C is now able to learn w' and therefore also $e_A(w)$.
- C now sends to Alice the pair $(e_A(e_A(w)\,C), A)$.
- Alice acknowledges by sending the pair $(e_C(e_C(w)\,A), C)$.
- C is now able to learn w.

Probabilistic signature schemes

Let us have a trapdoor permutation

$$f : D \to D,\ D \subset \{0,1\}^n,$$

a pseudorandom bit generator

$$G : \{0,1\}^l \to \{0,1\}^k \times \{0,1\}^{n-(l+k)},\quad w \to (G_1(w), G_2(w))$$

and a hash function

$$h: \{0,1\}^* \to \{0,1\}^l.$$

The following PSS scheme is applicable to messages of arbitrary length.

Signing: of a message $w \in \{0,1\}^*$.

1. Choose random $r \in \{0,1\}^k$ and compute $m = h(w \,||\, r)$.
2. Compute $G(m) = (G_1(m), G_2(m))$ and $y = m \,||\, (G_1(m) \oplus r) \,||\, G_2(m)$.
3. Signature of w is $\sigma = f^{-1}(y)$.

Verification of a signed message (w, σ).

- Compute $f(\sigma)$ and decompose $f(\sigma) = m \,||\, t \,||\, u$, where $|m| = l$, $|t| = k$ and $|u| = n - (k+l)$.
- Compute $r = t \oplus G_1(m)$.
- Accept signature σ if $h(w \,||\, r) = m$ and $G_2(m) = u$; otherwise reject it.

IV054 Authenticated Diffie-Hellman key exchange

Let each user U have a signature function s_U and a verification algorithm v_U.

The following protocol allows Alice and Bob to establish a key K to use with an encryption function e_K and avoids the man-in-the-middle attack.

1. Alice and Bob choose large prime p and a generator $q \in Z_p^*$.
2. Alice chooses a random x and Bob chooses a random y.
3. Alice computes $q^x \bmod p$, and Bob computes $q^y \bmod p$.
4. Alice sends q^x to Bob.
5. Bob computes $K = q^{xy} \bmod p$.
6. Bob sends q^y and $e_K(s_B(q^y, q^x))$ to Alice.
7. Alice computes $K = q^{xy} \bmod p$.
8. Alice decrypts $e_K(s_B(q^y, q^x))$ to obtain $s_B(q^y, q^x)$.
9. Alice verifies, using an authority, that v_B is Bob's verification algorithm.
10. Alice uses v_B to verify Bob's signature.
11. Alice sends $e_K(s_A(q^x, q^y))$ to Bob.
12. Bob decrypts, verifies v_A, and verifies Alice's signature.

An enhanced version of the above protocol is known as Station-to-Station protocol.

IV054 Security of digital signature

It is again very non-trivial to define security of digital signature.

Definition A chosen message attack is a process which on input of a verification key can obtain signature (corresponding to the given key) to message of its choice.

A chosen message attack is considered to be successful (in so called existential forgery) if it outputs a valid signature for a message for which it has not requested a signature during the attack.

A signature scheme is secure (or unforgeable) if every feasible chosen message attack succeeds with at most negligible probability.

Cryptographic protocols

Lecture 4: Digital signatures

Properties of signatures

- **Signature is authentic**: convinces recipient that the signer deliberately signed the document
- **Signature is unforgeable**: the signer, nobody else, deliberately signed the document
- **Signature is not reusable**: part of the document, can not be moved to a different document
- **Signed document is unalterable**: after signing, document can not be altered
- **Signature can not be repudiated**: The signer can not later claim he/she did not sign the document

Breaking a signature scheme

- Total break: an adversary is either able to compute the private key of the signing or finds an efficient signing algorithm equivalent to a valid signing algorithm.
- Selective forgery: An adversary is able to create a valid signature for a particular message or class of messages chose a priori.
- Existential forgery: An adversary is able to forge a signature for at least one message with little or no control over the message whose signature is obtained.

Attacks against signature schemes

- Key-only attack: adversary only knows the signer's public key
- Message attacks: Adversary is able to examine signatures corresponding either to known or chosen messages:
  - Known-message attack
  - Chosen-message attack (non-adaptive)
  - Adaptive chosen-message attack

Signing with symmetric cryptography

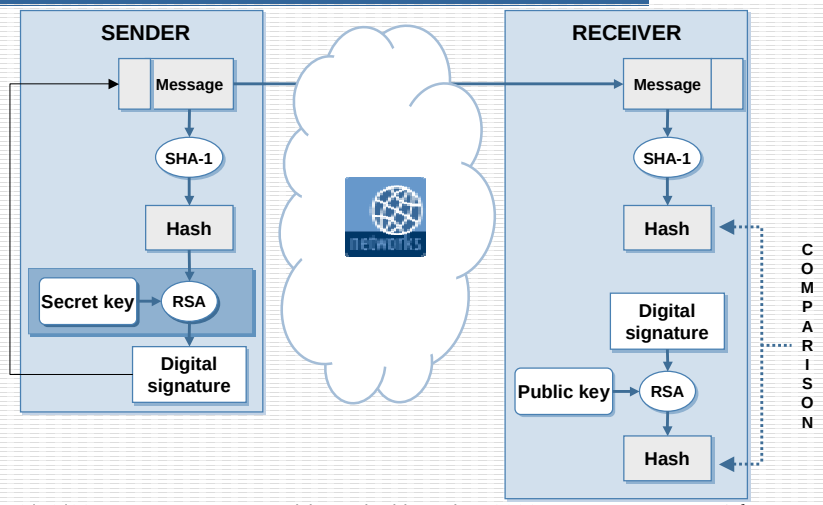
- Requires an arbitrator (T):

1. $A \rightarrow T: \quad E_{K_A}(m)$
2. $B \leftarrow T: \quad E_{K_B}(m, \text{"received from A"})$
3. $B: \quad \text{recovers } m, \text{ assures } m \text{ sent by } A$

Things you already know

- Arbitrators not practical in the real world
- Asymmetric cryptography makes signing feasible without arbitrator
- Usually, only a hash (e.g. 160-bit) signed instead of the entire document
- Timestamps often included in signed messages to prevent recipient cheating
- Non-repudiation requires secure storage of private keys (though is a legal, not technical, property of a signature)

Signing with asymmetric crypto



More formally

- A signs message *m*, B verifies the signature

1. $A: \quad H_m = h(m)$
2. $A: \quad \delta_m = S_A(H_m)$
3. $A \rightarrow B: \quad m, \delta_m$
4. $B: \quad H'_m = h(m)$
5. $B: \quad \delta_m \overset{?}{=} V_A(H'_m)$

Multiple signatures

- A and B want to sign the same document *m*:

1. $A:\ \delta_A = S_A(hash(m))$
2. $B:\ \delta_B = S_B(hash(m))$
3. $A \leftarrow B:\ \delta_B$
4. $A \rightarrow C:\ m, \delta_A, \delta_B$
5. $C:$ verifies δ_A and δ_B

group signatures

- Properties of group signatures:
  - Only members of a group can sign messages
  - The receiver from the signature can verify that it is a valid signature from a group
  - The receiver of the signature can not determine which member of the group is the signer
  - In case of a dispute, the identity of the original signer can be revealed

Group signature protocol

1. T: $K = \{(e_{1,1}, d_{1,1}), (e_{1,2}, d_{1,2}), \ldots, (e_{1,m}, d_{1,m}), \ldots (e_{n,1}, d_{n,1}), \ldots, (e_{n,m}, d_{n,m})\}$
2. $T \rightarrow G_i$: $K_i = \{e_{i,1}, \ldots e_{i,m}\}$ for each group member
3. T: publish public keys (not identities) $K_d = \{d_{1,1}, d_{1,2}, \ldots, d_{n,m}\}$
4. G_i: To sign a document, choose randomly $e' \in K_i$
5. V: Seek corresponding d from the public list, verify signature
6. T: Knows the signer's identity in case of a dispute

Blind signatures

- A sends some information to B for signing, B signs it and returns the signature to A
- From that signature, A can compute B's signature to arbitrary message *m*
- B will know neither *m* nor the signature of *m*
- B can not observe the message signed and later associate it to A
- Applications in e.g. anonymous eCash

Blind signature requirements

- Digital signing mechanism for B, i.e. a signing function $S_B(x)$ for any message x
- Blinding and unblinding functions f and g such that $g(S_B(f(m)))=S_B(m)$. $f(m)$ is called blinded message.
- E.g. blinded RSA
  - let k be a fixed integer gcd(n,k)=1, blinding function is $f(m)=mk^e \bmod n$. unblinding function is $g(m)=k^{-1}m \bmod n$

$$g(S_B(f(m))) = g(S_B(mk^e \bmod n)) = g(m^d k \bmod n) = m^d \bmod n = S_B(m)$$

Chaum's blind signature protocol

- A receives from B the signature of a blinded message, computes from it B's signature for an arbitrary message (selected by A) m. B has no knowledge of m or the signature. s is B's signature on m.

1. $A: \quad k = RND, \quad 1 \le k \le n-1, \gcd(k,n) = 1$
2. $A \rightarrow B: \quad m^* = mk^e \bmod n$
3. $A \leftarrow B: \quad s' = (m^*)^d \bmod n$
4. $A: \quad s = k^{-1}s' \bmod n$

Undeniable signatures

- Signature schemes where co-operation of the signer is required in verification
- For example, when a signer requires that the verifier can not proof the signer's actions without the signer's consent
- For example, Chaum-van Antwerpen undeniable signature scheme

Chaum-van Antwerpen key gen.

- Each entity selects a private key and corresponding public key. A does the following:

1. Select a random prime $p = 2q + 1$ where q is a prime
2. Select a generator α for the subgroup of order q in Z_p^*
   a. Select a random element $\beta \in Z_p^*$, compute $\alpha = \beta^{(P-1)/q} \bmod p$
   b. If $\alpha = 1$ then goto (a.)
3. Select a random integer $a \in \{1,2,\ldots,q-1\}$, compute $y = \alpha^a \bmod p$
4. A's public key is (p, α, y), private key is a

Chaum-van Antwerpen signing

- A signs message *m* so that B can verify the signature with co-operation of A
  1. $A: \quad s = m^a \bmod p$
- Verification protocol:
  1. $B:$ select random integers $x_1, x_2 \in \{1,2,\ldots,q-1\}$
  3. $A \leftarrow B: \quad z = s^{x_1} y^{x_2} \bmod p$
  4. $A \rightarrow B: \quad w = (z)^{a^{-1}} \bmod p \ (\text{where } aa^{-1} \equiv 1 (\bmod p))$
  5. $B: \quad w' = m^{x_1} \alpha^{x_2} \bmod p$ and accept iff $w = w'$

Disavowal

- A can disavow a valid signature by
  1. Refusing to participate in the protocol
  2. Perform the verification protocol incorrectly
  3. Claim that the signature is fake even if it verifies correctly
- A disavowal protocol may be established to guard against (2) and (3)

Chaum-van Antwerpen disavowal protocol

1. B: Obtain A's public key (p, α, y), Select random integers $x_1, x_2 \in \{1,2,\ldots,q-1\}$
2. $A \leftarrow B$: $z = s^{x_1} y^{x_2} \bmod p$
3. $A \rightarrow B$: $w = (z)^{a^{-1}} \bmod p$
5. B: $if\ (w = m^{x_1}\alpha^{x_2} \bmod p)$ Accept, halt
6. B: Select random integers $x'_1, x'_2 \in \{1,2,\ldots,q-1\}$
7. $A \leftarrow B$: $z' = s^{x'_1} y^{x'_2} \bmod p$
8. $A \rightarrow B$: $w' = (z')^{a^{-1}} \bmod p$
9. B: $if\ (w' = m^{x'_1}\alpha^{x'_2} \bmod p)$ Accept, halt
10. B: $c = (w\alpha^{-x_2})^{x'_1} \bmod p, \quad c' = (w'\alpha^{-x'_2})^{x_1}$

- If c=c', B concludes s is forgery, otherwise B concludes s is valid, A attempts to disavow it

Other signature schemes

- Proxy signatures
- Fail-stop signatures
- Designer confirmer signatures
- Many others

Thank you!

Jussipekka Leiwo, Ph.D.